

# The Hedged Random Forest

Elliot Beck<sup>1, 3</sup>, Damian Kozbur<sup>2</sup>, and Michael Wolf<sup>2, 4</sup>

<sup>1</sup>Department of Finance, University of Zurich

<sup>2</sup>Department of Economics, University of Zurich

<sup>3</sup>Swiss National Bank, Zurich

<sup>4</sup>ADIA Lab, Abu Dhabi

## Abstract

Over the past two decades, machine learning has become a central tool for applied researchers in both academia and industry. Among its most widely used methods is the random forest, applied to classification and regression tasks alike. This paper focuses on the regression setting, aiming to improve the accuracy of forecasts for a numeric response variable. Whereas the standard random forest aggregates tree-based forecasts with equal weights, we propose a more flexible weighting scheme inspired by financial portfolio-selection theory—one that, notably, permits negative weights. Using a benchmark collection of real-world datasets, we demonstrate that our approach enhances forecasting accuracy not only relative to the standard random forest but also relative to existing weighted-random-forest variants. Notably, the proposed methodology for weighting tree-based forecasts is general and can be applied in other forecast-combination problems as well.

KEY WORDS: Forecast combinations, machine learning, nonlinear shrinkage.

JEL classification codes: C21, C53.

# 1 Introduction

Machine learning (ML) methods have advanced substantially over the past two decades. As a result, they have become increasingly popular among applied researchers in economics and finance, helping to address a wide range of empirical challenges. Whereas individual studies on ML methods in (financial) economics are too numerous to list, comprehensive reviews are provided by Varian (2014), Mullainathan and Spiess (2017), and Athey and Imbens (2019). Additionally, the following text books provide fundamental treatments of ML methods: Hastie et al. (2017), Dixon et al. (2020), López de Prado (2020), Chan and Mátyás (2022), and Kelly and Xiu (2023).

Most ML applications in (financial) economics involve the modeling respectively forecasting of a response (or output) variable based on set of regressor (or input) variables, which is also known as *supervised learning*. One of the most popular ML methods to this end is the *random forest* (Breiman, 2001), given its ease of use, simplicity, and forecasting accuracy across a wide variety of applications. Depending on the nature of the response variable, the random forest can be used for either *classification* or *regression*: If the response variable is categorical, the random forest is used for *classification*; if the response variable is numerical, the random forest is used for *regression*. In this paper, the focus will be on regression, where the goal is to make accurate forecasts of the response variable.

It is useful to point out that ML methods, such as the random forest, can also be used for many other applications in (financial) economics apart from forecasting, even though such applications will not be pursued in this paper. One example is integrating ML methods into instrumental-variables frameworks as first-stage predictors; for example, see Belloni et al. (2012) and Hansen and Kozbur (2014). A second example is using ML methods to capture the effect of important conditioning information in causal-inference problems; for example, see Belloni et al. (2014) and Chernozhukov et al. (2018). A third example is employing ML methods to nonparametrically model treatment-effect heterogeneity; for example, see Athey et al. (2019) or Hansen et al. (2023).

Although there exist a large number of ML methods that can be used for forecasting in a regression setting, such as regularized linear regression (Lasso, Ridge, Elastic Net), kernel-smoothing methods, regression splines, and neural networks, Grinsztajn et al. (2022) demonstrate that tree-based methods, such as the random forest, continue to excel as top-performing ML forecasting methods on medium-sized datasets, which are still the norm in (financial) economics.

A key advantage of the random forest compared to standard linear regression models is that it can handle nonlinear relationships of unknown nature. Linear regression models also can handle nonlinear relationships but this involves (i) transformations of the response

variable or (some of) the regressors or (ii) inclusion of additional regressors such as powers or cross-products of original regressors. This avenue thus requires modeling choices and can also lead to the “curse of dimensionality”, for example, when all cross-products of original regressors are included. In contrast, the random forest requires no modeling choices and makes do with the collection of original regressors.

In the standard implementation of the random forest, one first grows an ensemble of (de-correlated) trees and then uses the simple average of the trees (also called the equal-weighted ensemble). This means that the individual forecasts of the trees are simply averaged to arrive at a final, combined forecast. For a textbook treatment on the random forest the reader is referred to Hastie et al. (2017, Chapter 15), for example.

The goal of this paper is to deviate from equal weighting in a data-dependent fashion that improves forecasting accuracy, on balance. One can consider this task to be a special case of the general, well-studied problem of *forecast combinations*. In this general problem, one has a fixed number of forecasting (or predictive) methods available and then assigns individual weights to them to obtain a combined forecast; these weights generally are constrained to sum to one, though there are exceptions to this convention. There exists an extensive literature on forecast combinations; for example, see Elliott and Timmermann (2016, Chapter 14), Wang et al. (2022), and the references therein.

We propose a high-level methodology for the general forecast-combination problem and then examine its use and performance in the specific context of combining tree-based forecasts within the random-forest framework. The high-level formulation of the problem is closely related to the well-studied portfolio-selection problem in finance. This connection allows us to draw on ideas from that literature. In this sense, our contribution may be considered “returning the favor”: Whereas much previous research has shown that machine learning can enhance portfolio selection, we now demonstrate—arguably for the first time—that portfolio selection can enhance machine learning. For reasons that will become clear below, we refer to the general approach as *hedging forecast combinations* and to its application to tree-based forecasts as the *hedged random forest*.

The remainder of the paper is organized as follows. Section 2 presents the general, high-level methodology which can be applied to a generic forecast-combination problem. Section 3 specializes to tree-based forecasting methods in the context of the random forest, giving specific guidance to implementation details. Section 4 provides an empirical application to real data, comparing the forecasting performance of our proposal with the standard random forest and previous proposals for weighted random forests. An appendix contains various robustness checks.

## 2 Methodology

### 2.1 General Description

This section details our high-level methodology which can be applied to a generic forecast-combination problem. The subsequent section then specializes to the problem of combining tree-based forecasts as an alternative the standard random forest, which simply assigns equal weight to all trees.

The goal is to forecast (or predict)<sup>1</sup> a random variable  $y \in \mathbb{R}$  based on a set of variables (or features)  $x \in \mathbb{R}^d$ . Denote a generic forecast by  $\hat{f}$ . Then its mean-squared error (MSE) is given by

$$\text{MSE}(\hat{f}) := \mathbb{E}(y - \hat{f}(x))^2 .$$

Here and below, the moments are, of course, obtained under the joint distribution governing the random vector  $v := (y, x')' \in \mathbb{R}^{1+d}$  and assumed to exist. Letting

$$\text{Bias}(\hat{f}) := \mathbb{E}(y - \hat{f}(x)) \quad \text{and} \quad \text{Var}(\hat{f}) := \mathbb{V}\text{ar}(y - \hat{f}(x)) = \mathbb{E}((y - \hat{f}(x))^2) - \text{Bias}^2(\hat{f}) .$$

there exists the well-known decomposition

$$\text{MSE}(\hat{f}) = \text{Bias}^2(\hat{f}) + \text{Var}(\hat{f}) . \tag{2.1}$$

The oracle that minimizes MSE is given by the conditional expectation  $\hat{f}_{\text{or}}(x) := \mathbb{E}(y|x)$ ; for example, see Hayashi (2000, Section 2.9). However, the oracle being an oracle, it is not available in practice.

We assume that a set (or universe) of  $p$  forecasting methods is available, denoted by  $\{\mathcal{M}_j\}_{j=1}^p$ . The number of methods,  $p$ , is assumed to be exogenous and fixed. Although we do not make this explicit in the notation, methods may be data-dependent in the sense that certain parameters may be fitted based on observed data; for example, the methods may correspond to linear regression models where each model postulates the identity (and the number) of regressors but not the values of the corresponding coefficients.

There exists an extensive literature on forecast combinations; for example, see Elliott and Timmermann (2016, Chapter 14), Wang et al. (2022), and the references therein. The consensus seems to be the equal weighting, given by

$$\hat{f}_{\text{EW}}(x) := \frac{1}{p} \sum_{j=1}^p \mathcal{M}_j(x) ,$$

---

<sup>1</sup>In this paper, the terms “forecast” and “predict” are used interchangeably; some people might associate a time-series setting with the term “forecast”, but we do not.

is hard to beat by more general linear combinations of the kind

$$\hat{f}_w(x) := \sum_{j=1}^p w_j \mathcal{M}_j(x) \quad \text{with} \quad w := (w_1, \dots, w_p)' \quad \text{and} \quad \sum_{j=1}^p w_j = 1 . \quad (2.2)$$

(The constraint that the weights sum to one is not necessary from a mathematical point of view, but it is natural and commonly used in the forecast-combination literature.) Our aim is to find a method to select a set of weights  $w$  that improves the MSE of equal weighting, on balance.

Denote by  $e_j := y - \mathcal{M}_j(x)$  the forecast error corresponding to model  $\mathcal{M}_j$  and collect these errors into the vector  $e := (e_1, \dots, e_p)'$  with expectation (vector)  $\mu := \mathbb{E}(e)$  and covariance matrix  $\Sigma := \text{Var}(e)$ . The MSE of a forecast of the type (2.2) is then given by

$$\text{MSE}(\hat{f}_w) = (w'\mu)^2 + w'\Sigma w .$$

(Note that the weights need to sum to one for this relation to hold.)

Therefore, the optimal (in terms of the MSE) forecast of the type (2.2) is the solution to the following optimization problem:

$$\min_w (w'\mu)^2 + w'\Sigma w \quad (2.3)$$

$$\text{s.t.} \quad w'\mathbb{1} = 1 , \quad (2.4)$$

where  $\mathbb{1}$  denotes a conformable vector of ones. If  $\Sigma$  is symmetric and positive-definite, Problem (2.3)–(2.4) is a convex optimization problem and can be solved easily and quickly with readily available software, even for large dimensions  $p$ .

The limitation in practice is that the inputs  $\mu$  and  $\Sigma$  are unknown. A feasible solution is to replace them with sample-based estimates  $\hat{\mu}$  and  $\hat{\Sigma}$ , which is an application of the general *plug-in principle*.

Being agnostic, for the time being, about the nature of the estimators  $\hat{\mu}$  and  $\hat{\Sigma}$ , we then solve the feasible optimization problem

$$\min_w (w'\hat{\mu})^2 + w'\hat{\Sigma}w \quad (2.5)$$

$$\text{s.t.} \quad w'\mathbb{1} = 1 \quad \text{and} \quad (2.6)$$

$$\|w\|_1 \leq \kappa , \quad (2.7)$$

where  $\|w\|_1 := \sum_{j=1}^p |w_j|$  denotes the  $L_1$  norm of  $w$  and  $\kappa \in [1, \infty]$  is a constant chosen by the user. If the estimator  $\hat{\Sigma}$  is symmetric and positive semi-definite, the optimization problem (2.5)–(2.7) is still of convex nature and can be solved easily and quickly, even for

large dimensions  $p$ . The solution to this optimization problem is denoted by  $\hat{w}$ .

The addition of the constraint (2.7) is motivated by the related problem of *portfolio selection* in finance, in which context the constraint is called a “gross-exposure constraint”. Adding this type of constraint to the infeasible problem (2.3)–(2.4) clearly would result in a (weakly) worse solution for any value  $\kappa \in [1, \infty)$ . But in the feasible problem, which must use estimated instead of true inputs, the constraint typically helps. The intuition here is that replacing  $\mu$  and  $\Sigma$  with respective estimates  $\hat{\mu}$  and  $\hat{\Sigma}$  can lead to unstable and underdiversified solutions that look good in sample (or in the training set) but perform badly out of sample, especially when the number of models,  $p$ , is not (exceedingly) small relative to the sample size relevant to the estimation of  $\mu$  and  $\Sigma$ ; for example, see Jobson and Korkie (1980), Michaud (1989), Jagannathan and Ma (2003). The addition of the gross-exposure constraint is particularly helpful when  $\hat{\Sigma}$  is given by the sample covariance matrix, an estimator that is unbiased but contains too much estimation error (unless  $p \ll n$ ). But even when sophisticated estimators of  $\Sigma$  are used, such as estimators based on factor models or shrinkage estimators, it can still be helpful to employ a gross-exposure constraint; for example, see Ding and Zheng (2025).

In the extreme case  $\kappa = 1$ , the weights are forced to be non-negative, that is,  $w_j \geq 0$ , which is called a “no-short-sales constraint” in finance. Imposing this constraint is standard in the forecast-combination literature, but it might well lead to sub-optimal performance because it does not give enough flexibility to the solution of the problem (2.5)–(2.7). At the other end of the spectrum, the choice  $\kappa = \infty$  corresponds to effectively removing the constraint (2.7), which may also lead to sub-optimal performance for the reasons mentioned above. Staying away from either extreme, there is ample evidence in the finance literature that choosing  $\kappa \in [1.5, 2.5]$  typically results in improved forecasting accuracy—and that the exact choice in this interval is not critical; for example, see DeMiguel et al. (2009).

Because the constraint (2.7) protects the user against extreme “positions”, that is, against weights  $\hat{w}_j$  that are unduly large in absolute value, we call our approach “hedging forecast combinations”.<sup>2</sup>

The notion of “hedging forecast combinations” should not be confused with the notion of “hedging predictions (or forecasts)” themselves. The latter notion means that one attaches to a prediction a quantitative measure of its own accuracy, such as a prediction interval; see Gammerman and Vovk (2007).<sup>3</sup>

---

<sup>2</sup>For example, according to Merriam-Webster (online version) the verb “to hedge against” means “to protect oneself from (something)”.

<sup>3</sup>We are grateful to Drago Indjić for pointing out this reference.

## 2.2 Theory

The solution to the convex optimization problem (2.5)–(2.7) is continuous in the inputs  $\hat{\mu}$  and  $\hat{\Sigma}$ . Therefore, with the choice  $\kappa = \infty$ , its solution  $\hat{w}$  leads to an asymptotically optimal forecast combination  $\hat{f}_{\hat{w}}$  based on consistent estimators  $\hat{\mu}$  and  $\hat{\Sigma}$ . Stating this fact as a formal result is possible, but this is a routine matter and has been recognized before. Furthermore, in practical applications, the relevant property is the finite-sample performance of the forecast  $\hat{f}_{\hat{w}}$  and, so far, the evidence based on simulation studies and empirical applications to real-life datasets indicates that such forecast combinations, on balance, do not outperform  $\hat{f}_{\text{EW}}$ , that is, equal weighting; again, see Elliott and Timmermann (2016, Chapter 14), Wang et al. (2022) and the references therein.

Our preceding high-level description is agnostic about the estimation of the mean  $\mu$  and the covariance matrix  $\Sigma$  of the vector of forecast errors  $e \in \mathbb{R}^p$ . In practice, we assume the existence of a training dataset  $\{v_i\}_{i=1}^n$  with  $v_i := (y_i, x_i')'$ . First, a set of (pseudo) forecast errors is constructed that are then used as input for the estimation of  $\mu$  and  $\Sigma$ . How one goes about this “best” in practice depends on both the nature of the data and the nature of the individual forecasting methods.

Consider the case where  $\{v_i\}_{i=1}^n$  is an independent and identically distributed (i.i.d.) sample whose distribution is equal to that of  $v$ . In this case, there exists a well-established literature on how to generate (pseudo) out-of-sample forecast errors, with a popular technique being cross-validation; for example, see Efron and Hastie (2022, Chapter 12) and Hastie et al. (2017, Chapter 7). Another option is to use in-sample errors, or residuals. This option has a bad reputation with some researchers, since in-sample errors tend to have a smaller variance than out-of-sample errors because of the well-known phenomenon of “overfitting”. However, for our purposes, this is not a serious problem; see Remark 2.1 below. In addition, it also has been argued that using in-sample errors can actually be advantageous relative to using (pseudo) out-of-sample errors; for example, see Hansen and Timmermann (2015).

The set of (pseudo) forecast errors then forms the basis for estimating  $\mu$  and  $\Sigma$ . To this end, one can use sample counterparts (that is, sample mean and sample covariance matrix), shrinkage methods, penalized estimation schemes, etc. Here, we restrict attention to shrinkage methods as an alternative to sample counterparts. For shrinkage estimation of a mean vector, see Hansen (2016), Bodnar et al. (2019), and the references therein; for shrinkage estimation of a covariance matrix, see Ledoit and Wolf (2022a) and the references therein.

Other data settings are also possible, for example, when  $\{v_1, \dots, v_n, v\}$  is a stationary time series. In this case also, the recommendation is to base the estimation of  $\mu$  and  $\Sigma$  on (pseudo) out-of-sample or in-sample forecast errors. How to generate those is less well

established compared to the i.i.d. case, but proposals do exist; for example, see Bergmeir and Benítez (2012) and Bergmeir et al. (2018). If one prefers shrinkage estimation over sample counterparts, one should use methods designed for time-series data; for example, see Sancetta (2008) and Engle et al. (2019) for shrinkage estimation of the covariance matrix. In contrast to i.i.d. data, where often “one-size-fits-all” solutions exist, time-series data generally require “custom-tailored” solutions that depend on the nature, frequency, and sample size of the available training data.

**Remark 2.1** (Scale invariance). The solution  $\hat{w}$  to the optimization problem (2.5)–(2.7) remains unchanged if  $\hat{\mu}$  and  $\hat{\Sigma}$  are replaced by  $c\hat{\mu}$  and  $c^2\hat{\Sigma}$ , respectively, for any constant  $c \in (0, \infty)$ . Therefore, it is not important that the estimators  $\hat{\mu}$  and  $\hat{\Sigma}$  get the levels of the true quantities  $\mu$  and  $\Sigma$  right. In particular, the use of in-sample (or training-set) errors in the construction of  $\hat{\mu}$  and  $\hat{\Sigma}$  can still lead to favorable performance of the forecast combination  $\hat{f}_{\hat{w}}$  even if such errors may have smaller variance than out-of-sample errors because of in-sample (or training-set) overfitting. Instead of approximating the actual entries of  $\mu$  and  $\Sigma$ , the corresponding estimators  $\hat{\mu}$  and  $\hat{\Sigma}$  only need to approximate the entries relative to each other in order for  $\hat{f}_{\hat{w}}$  to outperform  $\hat{f}_{\text{EW}}$ . ■

### 3 Hedging the Random Forest

Given the high-level description in the previous section, we only have to detail (i) how to compute forecast errors from training data and (ii) how to estimate  $\mu$  and  $\Sigma$  based on these errors.

We assume the existence of a training dataset  $\{v_i\}_{i=1}^n$  with  $v_i := (y_i, x_i')'$ . In this paper, we only consider the case where the  $\{v_i\}$  are i.i.d. with distribution identical to that of  $v$ . We train a random forest consisting of  $p$  trees on this dataset using the “ranger” package, a computationally efficient implementation of the random forest algorithm in the programming language R; see Wright and Ziegler (2017). A key hyperparameter in training a random forest is the number of features that are (randomly) selected from the universe of all  $d$  features at any given split. In “ranger”, this hyperparameter is called `mtry` and the default is  $\sqrt{d}$  for both tasks, classification and regression. However, in line with most of the literature, such as Liaw and Wiener (2002) and Hastie et al. (2017, Chapter 15), we set `mtry` equal to  $d/3$  for our task of regression.<sup>4</sup> This choice also aligns with the original work of (Breiman, 2001), who finds that higher `mtry` values are beneficial for the task of regression relative to the task of classification. All other hyperparameters are set to their default values, as recommended by Wright and Ziegler (2017) for the “ranger” package; in particular, we use

---

<sup>4</sup>If need be, values of `mtry` are rounded down to the nearest integer by common convention.



$p = 500$  trees.

After training the random forest, we extract the forecasts of each tree  $\mathcal{M}_j$  on the entire training set and thus obtain an in-sample error (or residual) matrix of size  $n \times p$ . We denote this matrix by  $R$  to indicate that it is not made up of actual forecast errors but of in-sample forecast errors (or residuals) instead. It is important to point out that we do not, for a given tree, extract forecasts on the corresponding out-of-bag observations only (that is, on the subset of the training set not used in growing the particular tree) because in this way we would not obtain a full  $n \times p$  matrix of residuals but instead a matrix containing a large number of missing values.<sup>5</sup> As explained below, we need a full matrix  $R$  for the estimation of  $\Sigma$ , if not for the estimation of  $\mu$  necessarily.

The various inputs to the feasible optimization problem (2.5)–(2.7) are chosen as follows. First, for the estimation of  $\mu$ , we use the (column-wise) sample mean of  $R$ ; we also experimented with some shrinkage estimators, but the results remained virtually unchanged. Second, for the estimation of  $\Sigma$ , we apply nonlinear shrinkage to  $R$ ; in particular, we use the quadratic inverse shrinkage (QIS) estimator of Ledoit and Wolf (2022b). Note here that nonlinear shrinkage requires a full matrix  $R$  as an input; a matrix with missing values would not be feasible. Third, for the gross-exposure constraint  $\kappa$  in (2.7), we use  $\kappa = 2$ . Appendix A runs robustness checks that consider (i) the sample covariance matrix rather than nonlinear shrinkage as the estimator  $\hat{\Sigma}$  and (ii) alternative values of the gross-exposure constraint  $\kappa$ .

All inputs to the optimization problem (2.5)–(2.7) are now in place. The solution  $\hat{w}$  assigns weight  $\hat{w}_j$  to tree  $\mathcal{M}_j$  rather than weight  $1/p$  as for the standard random forest (RF). We call this weighted random forest the *hedged random forest* (HRF). A corresponding software implementation in the programming language R can be found at <https://github.com/elliottbeck/hedgedrf>.

**Remark 3.1** (Absence of cross-validation). At this point, some readers may wonder why we do not use cross-validation to build a  $n \times p$  matrix of (pseudo) out-of-sample errors. The reason lies in the special nature of the random forest, since the individual forecasting methods, namely the trees, depend on the underlying training set. If we used ten-fold cross-validation, say, we would obtain ten different tree ensembles, none of which would coincide with the ensemble used at the end for forecasting, namely the ensemble based on the entire training set. This problem would not arise with other forecasting methods, such as linear regression models, say; of course, estimated parameters in a given regression model would change as a function of the underlying training set, but not the “characteristics” of that model, such as the identity (and the number) of the regressors.

---

<sup>5</sup>On average, there would be about  $1 - 1/e \approx 63.2\%$  of missing values

A leading motivation for the use of cross-validation is that the resulting (pseudo) out-of-sample errors have similar variance as actual out-of-sample errors, whereas in-sample errors tend to have smaller variance. However, as outlined in Remark 2.1, this is not a serious concern in our case. ■

**Remark 3.2** (Choice of nonlinear shrinkage estimator). There exist many nonlinear shrinkage formulas to estimate a covariance matrix; for example, see Ledoit and Wolf (2021). This is because one needs to base the optimal shrinkage formula on an underlying loss function, for which there exist many choices. Optimization problem (2.5)–(2.7) is related to Markowitz portfolio selection which suggests the use of the “minimum variance” loss function”; for example, see Ledoit and Wolf (2021, Table 1). The QIS estimator of Ledoit and Wolf (2022b) implements the corresponding shrinkage formula. ■

**Remark 3.3** (No need for user inputs). Importantly, hedging a random forest (with i.i.d. data) does not require user input, such as the choice of additional hyperparameters (beyond those that need to be chosen in the first place to train a random forest). In our experience, applied researchers prefer such “automatic” methods given that they generally lack the training or the time to select input parameters for machine learning methods. In this way, the hedged random forest also leads to reproducible results, which is another benefit. ■

**Remark 3.4** (Leverage constraint and solution density). Readers familiar with the Lasso may expect the gross-exposure constraint  $\|\mathbf{w}\|_1 \leq \kappa$  to induce a sparse solution, assigning nonzero weight to only a small number of trees. This intuition, however, does not carry over to the present setting because of the equality constraint  $\mathbf{1}^\top \mathbf{w} = 1$ . An  $L_1$  penalty alone encourages sparsity by pushing the solution to a vertex of the  $L_1$  ball, where all but a few coordinates are zero. The hyperplane  $\mathbf{1}^\top \mathbf{w} = 1$  generically cuts through the interior of that ball rather than through its sparse vertices, so the constrained optimum lies in the interior and is typically *dense*: all trees receive a nonzero weight, though some weights may be negative when  $\kappa > 1$ . The role of  $\kappa$  is therefore not to select a sparse subset of trees but to bound the overall leverage, in direct analogy with gross-exposure constraints in portfolio theory. ■

## 4 Empirical Applications

### 4.1 Data

In a first application, we use the 17 numerical regression benchmark datasets proposed by Grinsztajn et al. (2022) to evaluate the performance of the hedged random forest (HRF) in comparison with the standard equal-weighted random forest (RF).<sup>6</sup> The datasets are accessible on the official openML website [www.openml.org](http://www.openml.org), which also offers comprehensive metadata and descriptions; more than half of these datasets pertain to the areas economics and finance. To ensure the relevance and quality of these datasets, Grinsztajn et al. (2022) employ the following criteria.

- **Heterogeneous columns:** Columns correspond to features of different nature. For example, this criterion excludes images or signal datasets where each column corresponds to the same signal on different sensors, for example.
- **Not high dimensional:** Only datasets with a  $d/n$  ratio below 1/10 are included, where  $d$  denotes the numbers of features.
- **Well-documented datasets:** Only datasets with sufficient metadata are included.
- **I.I.D. data:** Stream-like datasets and time-series data sets are excluded.
- **Real-world data:** Artificial datasets are excluded.
- **Not too small:** Datasets with fewer than four features or fewer than 3,000 observations are excluded.
- **Not too easy:** For a dataset to be included, linear regression must perform “sufficiently worse” than certain machine-learning methods; for details, see Grinsztajn et al. (2022).
- **Not deterministic:** datasets where the response variable is a deterministic function of the features are excluded. This mostly means removing datasets on games such as poker and chess.
- **No missing data:** All observations that contain missing values are excluded from the datasets. No imputations are performed.

For large datasets, we follow Grinsztajn et al. (2022) by selecting 10,000 observations at random. To avoid interference with the data, we refrain from any attribute-preprocessing. Table 1 lists the datasets used in the empirical application together with the corresponding numbers of observations and numbers of features.

---

<sup>6</sup>It might be tempting to look for datasets where the weighting of trees is particularly effective relative to the standard random forest. But doing so would amount to data mining and result in an unfair comparison. By using a standard collection of datasets from the literature, we preempt any such concerns.

| dataset                 | # Observations | # Features |
|-------------------------|----------------|------------|
| ailerons                | 13,750         | 734        |
| Bike_Sharing_Demand     | 17,379         | 12         |
| brazilian_houses        | 10,692         | 10         |
| cpu_act                 | 8,192          | 22         |
| diamonds                | 53,940         | 10         |
| elevators               | 16,599         | 12         |
| house_16H               | 22,784         | 17         |
| house_sales             | 21,613         | 18         |
| houses                  | 20,640         | 9          |
| isolet                  | 7,797          | 614        |
| medical_charges         | 163,065        | 4          |
| MiamiHousing2016        | 13,932         | 14         |
| nyc-taxi-green-dec-2016 | 581,835        | 19         |
| pol                     | 15,000         | 49         |
| sulfur                  | 10,081         | 7          |
| superconduct            | 21,263         | 80         |
| wine_quality            | 6,497          | 12         |

Table 1: Datasets used.

## 4.2 Results

Any dataset is partitioned into a training set and a test set by selecting  $n$  observations at random, which constitute the training set, and then using the remainder of the observations as the test set. For each method, RF and HRF, fitted on the training set, we obtain a MSE on the test set, denoted by  $\text{MSE}_{\text{RF}}$  and  $\text{MSE}_{\text{HRF}}$ , respectively. In order to protect against spurious results due to random partitioning of the data, we then repeat this process (independently)  $B$  times. As the final performance measure, we report the following root-mean-squared-error (RMSE) ratio:

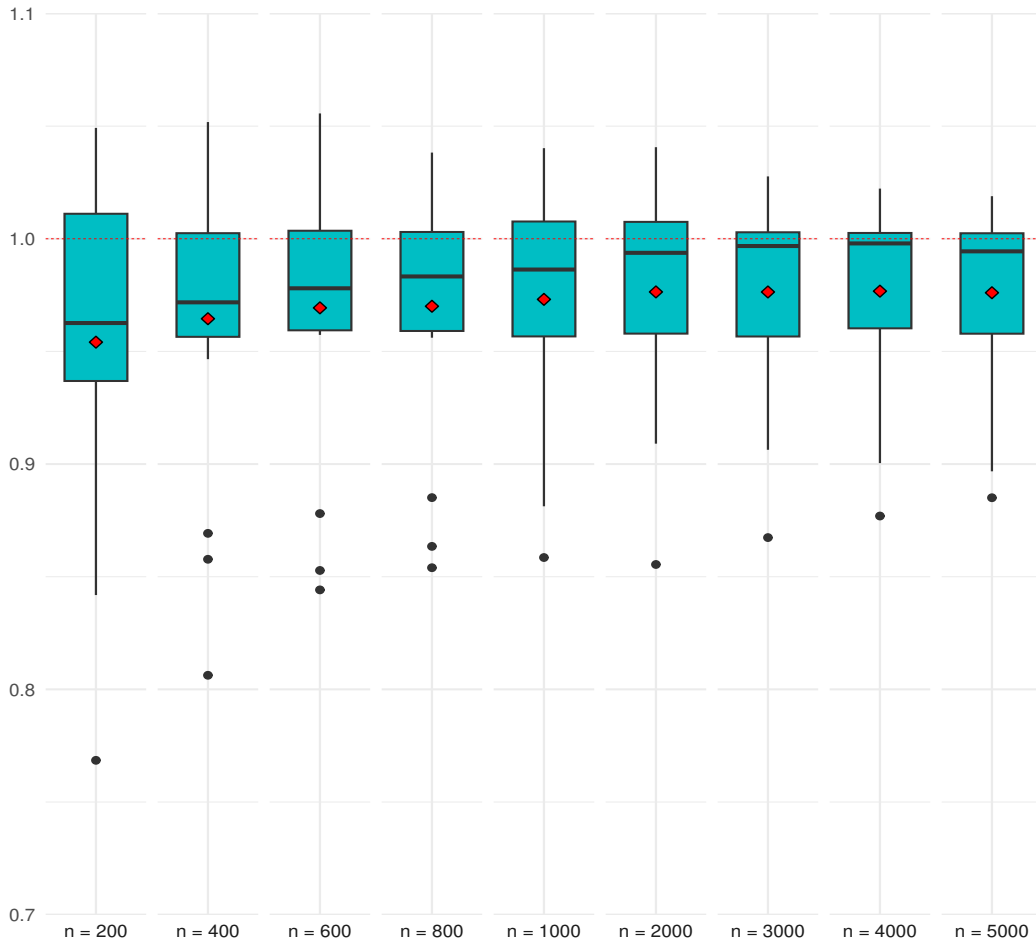
$$\text{RMSE}_{\text{HRF/RF}} := \frac{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{HRF},b}}}{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{RF},b}}} . \quad (4.1)$$

This means that for each method, we average the MSE values over the repetitions  $B$  and then take the root to arrive at the individual RMSE values. Finally, we take the ratio of the two individual RMSE values. Ratios greater than one speak in favor of RF, whereas ratios smaller than one speak in favor of HRF. Our results below are all based on  $B = 100$  repetitions; larger values of  $B$  leave the results virtually unchanged.

In this way, for any training-set size  $n$ , we obtain 17 RMSE ratios (4.1), one for each dataset listed in Table 1. We convert the 17 ratios into a boxplot and then line up the

boxplots for  $n \in \{200, 400, 600, 800, 1000, 2000, 3000, 4000, 5000\}$  in Figure 1. The results<sup>7</sup> can be summarized as follows:

- On balance, HRF clearly outperforms RF.
- The gains are most pronounced for small training-set sizes  $n$  but extend up to the largest size considered,  $n = 5000$ .
- Out of the 17 datasets, there are five on which HRF performs worse than RF, but the loss is never more than 8% and always below 5% for  $n \geq 600$ .
- On the other hand, there are two datasets for which HRF reduces the RMSE compared to RF by more than 10% for  $n \leq 1000$ ; for one of these datasets, HRF actually reduces the RMSE by more than 10% for all  $n$ .
- Summing up, based on the 17 datasets considered, there is little to lose but potentially much to gain by upgrading from RF to HRF.



<sup>7</sup>Appendix B details the individual RMSE results for all datasets and methods.

Figure 1: Boxplots of RMSE ratios (4.1). For each training-set size  $n$ , the boxplot is based on the 17 ratios corresponding to the datasets listed in Table 1.

### 4.3 Negative Weights and Shrinkage

All previous proposals for weighting the random forest we are aware of, not only in the context of regression but also in the context of classification, impose the “no-short-sales constraint”  $\kappa = 1$ , that is,  $w_j \geq 0$ . As shown in the robustness checks of Appendix A, allowing for negative weights typically improves the performance of HRF, and for certain datasets by a pronounced margin, when  $\Sigma$  is estimated with nonlinear shrinkage. In contrast, this is not the case when the sample covariance matrix is used to estimate  $\Sigma$ . To illustrate this finding visually, we consider the following two RMSE ratios:

$$\text{RMSE}_{\kappa=2/\kappa=1}^{\text{sample}} := \frac{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{HRF},b,\kappa=2,\text{sample}}}}{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{HRF},b,\kappa=1,\text{sample}}}} \quad (4.2)$$

and

$$\text{RMSE}_{\kappa=2/\kappa=1}^{\text{shrinkage}} := \frac{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{HRF},b,\kappa=2,\text{shrinkage}}}}{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{HRF},b,\kappa=1,\text{shrinkage}}}}. \quad (4.3)$$

These ratios measure the effect of allowing for negative weights ( $\kappa = 2$ ) relative to imposing non-negative weights ( $\kappa = 1$ ): Ratio (4.2) does so when  $\Sigma$  is estimated by the sample covariance matrix whereas ratio (4.3) does so when  $\Sigma$  is estimated by nonlinear shrinkage. Figure 2 shows that allowing for negative weights, in balance, leads to poorer performance when  $\Sigma$  is estimated by the sample covariance matrix—in contrast to using nonlinear shrinkage, where it leads to better performance.

This finding might explain to some extent why the forecast-combination literature, so far, has generally shied away from negative weights: It is still the *status quo* in this literature to use the sample covariance matrix, a practice that should be abandoned.

A portfolio-theoretic perspective helps clarify when negative weights are most beneficial. In the minimum-variance portfolio literature it is well known that optimal weights are all non-negative when the constituent assets are mutually uncorrelated: independent variance reduction can be achieved without any short positions. As pairwise correlations increase, however, negative weights become necessary to fully exploit diversification—the optimal portfolio “shorts” the most redundant assets to cancel shared noise. The same logic applies to random forest trees. Trees are explicitly decorrelated via feature subsampling (the `mtry` parameter), but this decorrelation is imperfect when the input features are themselves highly correlated. In such datasets trees remain substantially correlated, and allowing  $\kappa = 2$

is expected to yield the largest gains over  $\kappa = 1$ , because the negative weights cancel the systematic component of prediction error that the correlated trees share. Conversely, when features are nearly independent and trees are already well-decorrelated, the non-negativity constraint ( $\kappa = 1$ ) is approximately binding without penalty, and the additional flexibility of  $\kappa = 2$  adds little. This provides an interpretable characterisation of when the hedged random forest with negative weights is most valuable.

In real-life finance applications, a “no-short-sales constraint” can be motivated by legislation (for example, mutual funds are not allowed to short stocks) or by practical considerations (for example, shorting certain assets may not be possible or may be prohibitively expensive). On the other hand, “short-selling” individual forecasting methods  $\mathcal{M}_j$  by assigning them a negative weight does not violate any laws and does not incur any monetary costs. We therefore hope that this paper will serve as motivation to allow for negative weights not only in the random forest but also in other forecast-combination settings. Just remember to abandon the sample covariance matrix in favor of shrinkage estimation of  $\Sigma$ , unless the number of forecasting methods is negligible relative to the number of observations in the training set.

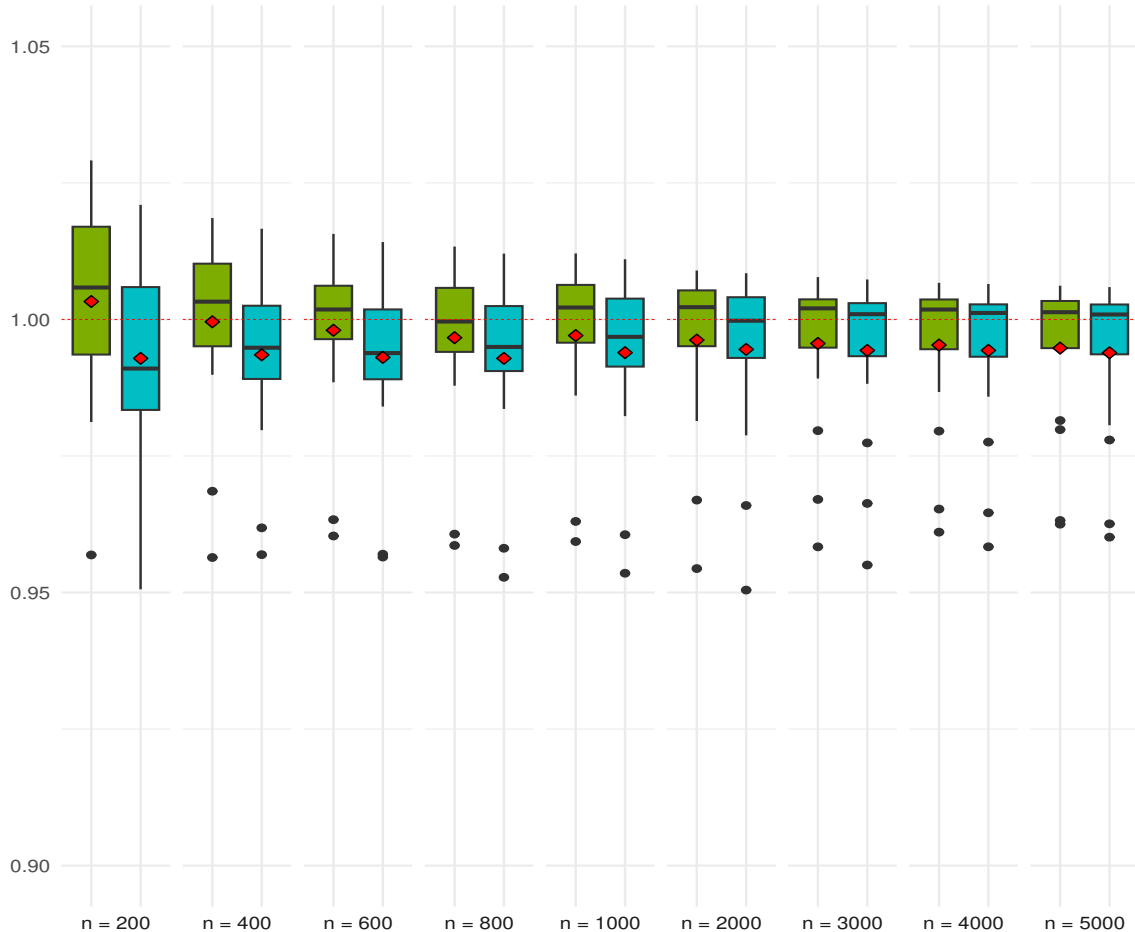


Figure 2: Boxplots of RMSE ratios (4.2) in green and RMSE ratios (4.3) in blue. For each training-set size  $n$ , the boxplots are based on the 17 ratios corresponding to the datasets listed in Table 1.

## 4.4 Related Literature

There have been previous proposals in the literature for weighting the random forest, so it is of interest to compare the HRF with them. [We also compare the HRF against general \(not random-forest-specific\) forecast-combination benchmarks and against a variant of the HRF’s own optimization problem.](#)

### 4.4.1 Comparison One

[In this comparison, we consider five alternative weighting or combination schemes for the random forest.](#) We first compare the HRF with two proposals originally intended for the task of classification but easily adaptable for the task of regression, namely the weighted random forest (WRF) of Winham et al. (2013) and the Césaro random forest (CRF) of Pham and Olafsson (2020). As will be seen below, the weights  $\hat{w}_j$  in both proposals are always non-negative; actually, they are always strictly positive.

For the weighted random forest of Winham et al. (2013), one first computes, for  $j = 1, \dots, p$ , the tree-accuracy measure

$$tPE_j := \frac{1}{|\text{OOB}_j|} \sum_{i \in \text{OOB}_j} |y_i - \mathcal{M}_j(x_i)|, \quad (4.4)$$

where  $\text{OOB}_j \subset \{1, \dots, n\}$  denotes the out-of-bag set corresponding to tree  $\mathcal{M}_j$ , that is, the subset of the training set not used in the training of the tree. Next, one computes relative weights according to one of the three following formulas:

$$\hat{w}_{j,\text{rel}} := 1 - tPE_j \quad (4.5)$$

$$\hat{w}_{j,\text{rel}} := \exp\left(\frac{1}{tPE_j}\right) \quad (4.6)$$

$$\hat{w}_{j,\text{rel}} := \left(\frac{1}{tPE_j}\right)^\lambda \quad \text{for some } \lambda > 0 \quad (4.7)$$

Finally, the weights  $\{\hat{w}_j\}$  are forced to sum up to one by setting

$$\hat{w}_j := \frac{\hat{w}_{j,\text{rel}}}{\sum_{l=1}^p \hat{w}_{l,\text{rel}}}.$$

Following the advice of Winham et al. (2013), we tried as the leading candidates version (4.6)



and version (4.7) with  $\lambda := 5$ , of which the former performed somewhat better and is therefore used below. We can construct as an analog to (4.1) the ratio

$$\text{RMSE}_{\text{HRF}/\text{WRF}} := \frac{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{HRF},b}}}{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{WRF},b}}} . \quad (4.8)$$

For the Césaro random forest of Pham and Olafsson (2020), one uses the tree-accuracy measures  $t\text{PE}_j$  defined in (4.4) to construct the weights, but in a different scheme: One sorts the measures  $\{t\text{PE}_1, \dots, t\text{PE}_p\}$  from smallest to largest and defines  $r_j$  as the order of  $t\text{PE}_j$  in the sorted sequence. Hence, if  $t\text{PE}_{(1)} \leq t\text{PE}_{(2)} \leq \dots \leq t\text{PE}_{(p)}$ , we have  $t\text{PE}_j = t\text{PE}_{(r_j)}$ . Then the relative Césaro weights are defined as

$$\hat{w}_{j,\text{rel}} := \sum_{l=r_j}^p \frac{1}{l} . \quad (4.9)$$

Finally, the weights  $\{\hat{w}_j\}$  are forced to sum up again to one by setting

$$\hat{w}_j := \frac{\hat{w}_{j,\text{rel}}}{\sum_{l=1}^p \hat{w}_{l,\text{rel}}} .$$

We can construct as an analog to (4.1) the ratio

$$\text{RMSE}_{\text{HRF}/\text{CRF}} := \frac{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{HRF},b}}}{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{CRF},b}}} . \quad (4.10)$$

As an analog to Figure 1, we now obtain Figure 3 which demonstrates that HRF clearly outperforms WRF, on balance. The comparison of HRF with CRF is somewhat more subtle: Although the two methods exhibit comparable performance in terms of the median RMSE ratio, HRF consistently outperforms CRF in terms of the mean RMSE ratio for all training-set sizes  $n$ . Hence, taking everything into account, HRF also outperforms CRF, on balance.<sup>8</sup>

We further benchmark HRF against three combination schemes that are not specific to random forests. *Ridge* estimates the weights  $w$  by ridge-penalized least squares of the training-set labels on the in-sample tree predictions, subject to  $w'1 = 1$ , with the ridge penalty chosen by AIC. *2-Step Ridge* uses the same estimator but, following the sample-splitting idea of Timmermann (2006), estimates  $w$  on a held-out second half of the training set rather than on the sample used to grow the trees, so as to avoid in-sample overfitting.

---

<sup>8</sup>Appendix B presents the RMSE results for all datasets and all methods considered (RF, HRF, WRF, CRF, Ridge, 2-Step Ridge, and HRF-MinVar).

*HRF-MinVar* solves the same optimization problem (2.5)–(2.7) underlying HRF but with the mean vector  $\hat{\mu}$  set to zero in the objective, that is, a pure variance-minimization problem; the (scalar) in-sample bias  $w'\hat{\mu}$  is then added back to the resulting forecast. Comparing HRF-MinVar with HRF thus isolates the incremental contribution of explicitly modeling the bias term in the mean-variance objective (2.5).

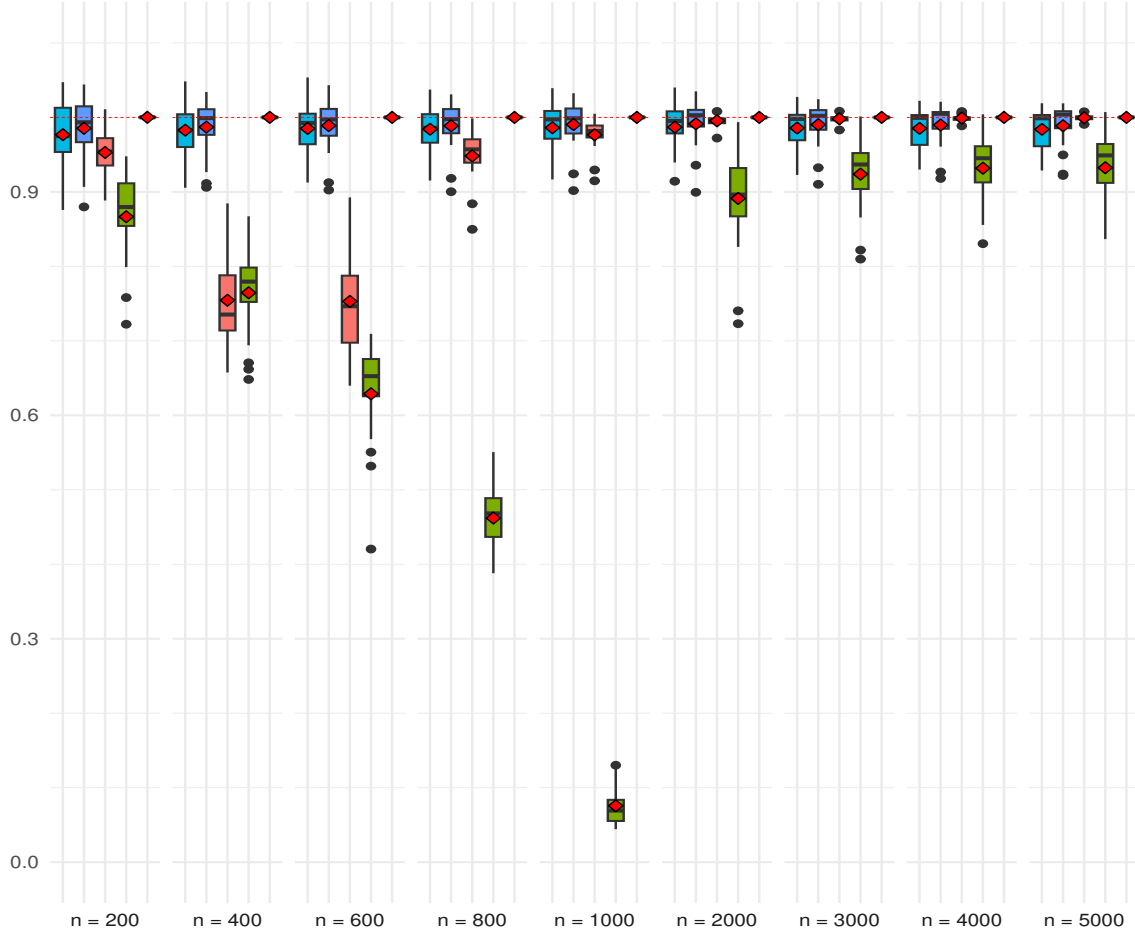


Figure 3: Boxplots of RMSE ratios (4.8) in light blue and RMSE ratios (4.10) in dark blue, together with the analogous RMSE ratios of HRF relative to Ridge (red), 2-Step Ridge (green), and HRF-MinVar (purple). For each training-set size  $n$ , the boxplots are based on the 17 ratios corresponding to the datasets listed in Table 1.

HRF-MinVar tracks HRF almost exactly (RMSE ratios between 0.999 and 1.001) at every training-set size, indicating that the variance term in the mean-variance objective (2.5) accounts for virtually all of the gains over the equal-weighted RF, while the explicit bias term contributes little once the weights are constrained as in (2.6)–(2.7). Ridge is a reasonably strong benchmark but is still dominated by HRF, on balance, at every training-set size. 2-Step Ridge, in contrast, becomes highly unstable whenever the training-set size is close to twice the number of trees  $p = 500$  (so that the second-stage regression is estimated

from about as many observations as there are trees to weight), causing its RMSE to spike relative to HRF; this instability is most pronounced at  $n = 1000$  and illustrates the benefit of estimating the covariance matrix  $\Sigma$  with nonlinear shrinkage on the full training sample, as HRF does, rather than resorting to sample splitting.

#### 4.4.2 Comparison Two

Next, we compare the HRF with the proposal of Chen et al. (2024), which has been designed for the task of regression and builds on the framework of Qiu et al. (2020). The latter paper introduces a method for combining the outputs of multiple machine learning algorithms (aka base learners) using weights derived from Mallows-type criteria. Chen et al. (2024) extend this idea to random forests by treating individual trees as base learners. They prove that, in a certain setting, their approach is asymptotically optimal and hence use the acronym  $\text{WRF}_{\text{opt}}$  which stands for “optimal weighted random forest”. Chen et al. (2024) propose two implementations: (i)  $\text{1step-WRF}_{\text{opt}}$ , a computationally intensive version, and (ii)  $\text{2steps-WRF}_{\text{opt}}$ , a computationally more efficient version, which actually performs better in their empirical application. Consequently, we adopt version  $\text{2steps-WRF}_{\text{opt}}$  for our comparison and refer to it as “optimal weighted random forest” (OWRF).

We originally wanted to include OWRF in the analysis of Section 4.4.1 but could not obtain results.  $\text{2steps-WRF}_{\text{opt}}$  is based on quadratic optimization which in practice is carried out with a numerical solver. But this solver does not converge reliably, regardless of whether one uses the hyperparameters for training the random forest as detailed in Section 3 or those proposed by Chen et al. (2024), which are detailed below. The problem remains if one uses  $\text{1step-WRF}_{\text{opt}}$  instead of  $\text{2steps-WRF}_{\text{opt}}$ .

To enable a comparison, we thus use the datasets selected by Chen et al. (2024), which amounts to giving them the “home field advantage”. This is because instead of using a benchmark collection of datasets from the literature, the authors select 12 datasets themselves (without providing the corresponding rationale). Specifically, Chen et al. (2024) select 11 datasets from the UCI Machine Learning Repository plus an additional dataset from `openml.org`. To implement OWRF, we use the code that has been made publicly available by the authors<sup>9</sup> and, surprisingly, differs in certain aspects from the description of the underlying algorithm in the paper. Following the code of Chen et al. (2024), each dataset is randomly partitioned into a training set (70%) and a testing set (30%).<sup>10</sup> In order to guard against spurious results due to the random partitioning of the data, this

<sup>9</sup>See <https://github.com/XinyuChen-hey/Optimal-Weighted-Random-Forests>.

<sup>10</sup>This is in contrast to the paper which states that each dataset is randomly partitioned into a training set (50%), a testing set (30%), and a validation set (20%).

process is then repeated (independently)  $B = 1,000$  times<sup>11</sup> and the performance measures are then averaged across repetitions, as described below.

Importantly, Chen et al. (2024) do not use the same hyperparameter choices as we (which are detailed in Section 3). To start with, they only use  $p = 100$  trees instead of  $p = 500$  trees; this may be because of the computational expense of their implementation 1step-WRF<sub>opt</sub>, which is also included in their empirical application. More importantly, each tree is only grown until a minimum of  $\sqrt{n}$  observations remain per leaf (with  $n$  denoting the number of training observations). This choice is nonstandard and results in much more “shallow” trees compared to standard conventions in the literature. When conventional choices for the minimum number of observations remaining per leaf are used, the OWRF algorithm does not even converge reliably on the 12 datasets selected by the authors themselves. To wit, Chen et al. (2024) use the same value for the hyperparameter `mtry` as we do, that is,  $d/3$  (rounded down to the nearest integer if necessary). Hence, the key difference is that they use “shallow” trees whereas we use “deep” trees.

In the comparison of forecasting methods, as in Chen et al. (2024), we include RF as well as WRF and CRF from Section 4.4.1. [That is, we use the same benchmarks as Chen et al. \(2024\), the paper that proposed this collection of datasets, and add our own method, HRF.](#) For all three methods, we retain the hyperparameter choices introduced in Section 3 for HRF, as this reflects how applied researchers would typically use these methods in practice. This is in contrast to Chen et al. (2024) who retain the OWRF hyperparameter also for RF, WRF, and CRF; this means that, essentially, they compare OWRF with methods that rarely would be used in practice.

Chen et al. (2024) evaluate forecasting performance using the following two measures:

$$\text{MSFE} := \frac{1}{B * n_{\text{test}}} \sum_{b=1}^B \sum_{i=1}^{n_{\text{test}}} (\hat{y}_{b,i} - y_i)^2 \quad \text{and} \quad \text{MAFE} := \frac{1}{B * n_{\text{test}}} \sum_{b=1}^B \sum_{i=1}^{n_{\text{test}}} |\hat{y}_{b,i} - y_i| ,$$

where  $B$  denotes the number of data-partition repetitions,  $\hat{y}_{d,i}$  denotes the forecast of  $y_i$  in the  $b^{\text{th}}$  repetition, and  $n_{\text{test}}$  denotes the sample size of the test data.

Table 2 presents the results: Panel (a) presents the MSFE results and Panel (b) presents the MAFE results. It can be seen that, in terms of either criterion, HRF stands out as the clear winner, providing the lowest value for 11 out of the 12 datasets.

Notably, the “optimal” method OWRF is best zero times and is not only outperformed by HRF (always) but also by WRF and CRF (on balance). The latter finding is in contrast to the results in Chen et al. (2024), the reason being that the authors retain the nonstandard OWRF hyperparameters also for WRF and CRF.

---

<sup>11</sup>Chen et al. (2024) use  $D$  instead of  $B$  to denote the number of repetitions.

| Panel (a): MSFE |       |              |       |             |       |
|-----------------|-------|--------------|-------|-------------|-------|
| Dataset         | RF    | HRF          | OWRF  | WRF         | CRF   |
| BH              | 11.66 | <b>10.26</b> | 12.01 | 11.00       | 11.09 |
| servo           | 1.49  | <b>0.60</b>  | 0.77  | 0.70        | 1.05  |
| CCS             | 32.47 | <b>25.39</b> | 39.35 | 29.00       | 29.45 |
| ASN             | 13.27 | <b>5.43</b>  | 12.30 | 8.96        | 10.31 |
| CCPP            | 11.56 | <b>11.14</b> | 14.85 | 11.31       | 11.22 |
| CST             | 18.74 | <b>12.10</b> | 15.57 | 13.74       | 15.69 |
| EE              | 2.33  | <b>1.88</b>  | 2.60  | 2.09        | 2.21  |
| PT              | 0.26  | <b>0.12</b>  | 0.61  | 0.16        | 0.19  |
| QSAR            | 1.25  | 1.28         | 1.34  | <b>1.23</b> | 1.24  |
| SM              | 2.46  | <b>0.88</b>  | 2.25  | 1.30        | 1.87  |
| YH              | 17.73 | <b>1.14</b>  | 2.54  | 1.68        | 5.53  |
| tecator         | 4.19  | <b>2.63</b>  | 3.28  | 2.65        | 3.09  |

| Panel (b): MAFE |      |             |      |             |      |
|-----------------|------|-------------|------|-------------|------|
| Dataset         | RF   | HRF         | OWRF | WRF         | CRF  |
| BH              | 2.25 | <b>2.16</b> | 2.36 | 2.20        | 2.21 |
| servo           | 0.87 | <b>0.45</b> | 0.54 | 0.49        | 0.71 |
| CCS             | 4.21 | <b>3.57</b> | 4.82 | 3.93        | 3.98 |
| ASN             | 2.94 | <b>1.75</b> | 2.76 | 2.35        | 2.55 |
| CCPP            | 2.47 | <b>2.40</b> | 2.92 | 2.44        | 2.42 |
| CST             | 3.26 | <b>2.62</b> | 3.00 | 2.80        | 3.00 |
| EE              | 0.97 | <b>0.83</b> | 1.05 | 0.90        | 0.94 |
| PT              | 0.34 | <b>0.23</b> | 0.57 | 0.26        | 0.29 |
| QSAR            | 0.82 | 0.83        | 0.85 | <b>0.81</b> | 0.81 |
| SM              | 1.20 | <b>0.68</b> | 1.13 | 0.84        | 1.03 |
| YH              | 2.61 | <b>0.54</b> | 0.97 | 0.70        | 1.47 |
| tecator         | 1.45 | <b>1.09</b> | 1.31 | 1.18        | 1.27 |

Table 2: Comparison of five forecasting methods (RF, HRF, OWRF, WRF, and CRF) using two performance criteria (MSFE and MAFE) across 12 datasets. Panel (a) presents MSFE results and Panel (b) presents MAFE results. In each row, the lowest (and thus best) value is in **bold face**.

**Remark 4.1** (Inability to reproduce exact results). The reader may observe that the OWRF results in Table 2 differ somewhat from the corresponding 2steps-WRF<sub>opt</sub> results in Tables 3 and 4 of Chen et al. (2024). All we can say is that we have faithfully executed the authors’ publicly available code. Why the results we obtain do not match those reported in the paper is unclear. ■

## 4.5 Additional Application to Finance

As an additional application to finance, we consider the “Black Friday Sales Prediction” challenge. The dataset is available at [www.kaggle.com/datasets/sdolezel/black-friday](http://www.kaggle.com/datasets/sdolezel/black-friday) and further information can be found at [www.analyticsvidhya.com/datahack/contest/black-friday](http://www.analyticsvidhya.com/datahack/contest/black-friday). The goal is to forecast the sales amount for a specific purchase on Black Friday based on 11 features, such as gender, age, occupation, and marital status of the purchaser, as well as city and product categories. The total number of observations in the dataset is over half a million, so we select 10,000 observations at random as in Section 4.1. Then we compute RMSE ratios (4.1) for various training-set sizes  $n$  as described in Section 4.2. Table 3 presents the results which demonstrate that HRF outperforms RF consistently for all training-set sizes  $n$ .

| $n$  | (4.1) |
|------|-------|
| 200  | 0.993 |
| 400  | 0.979 |
| 600  | 0.973 |
| 800  | 0.972 |
| 1000 | 0.971 |
| 2000 | 0.974 |
| 3000 | 0.975 |
| 4000 | 0.975 |
| 5000 | 0.977 |

Table 3: RMSE ratios (4.1) for various training-set sizes  $n$ .

## 4.6 Time-Series Data

We have previously demonstrated that the hedged random forest, on balance, provides superior forecasting performance relative to the standard random forest. The evidence has been for cross-sectional (or i.i.d.) datasets. Such datasets are easy to handle in the sense that we can offer “one-size-fits-all” estimators of the two inputs required for the hedged random forest, the mean vector  $\mu$  and the covariance matrix  $\Sigma$  of the vector of forecast

errors corresponding to the individual trees. These estimators do not require any user inputs and thus applied researchers only have to “point and shoot”.

Things change in the case of time-series data: What are suitable estimators of  $\mu$  and  $\Sigma$  depends on the nature and frequency of the data and on the sample size. Therefore, these estimators have to be custom-tailored to the application at hand. This is unfortunate but also “par for the course” for time-series applications where “one-size-fits-all” solutions rarely exist.

Beck and Wolf (2025) detail how to adapt the hedged random forest to the specific task of forecasting inflation and demonstrate that it consistently outperforms the standard random forest in terms of forecasting accuracy.

## 5 Conclusion

Machine learning (ML) methods have become an important empirical tool for virtually all empirically oriented researchers in economics and finance. In particular, the random forest is one of the most popular and widely employed supervised machine learning methods. It is especially suitable for applications in economics and finance, where datasets are generally still “small to medium sized” by machine learning standards; for example, see Grinsztajn et al. (2022) and Januschowski et al. (2022).

This paper has focused on the scenario where the random forest is used to forecast a numerical variable, that is, on the task of regression. In its standard form, the random forest is an equal-weighted ensemble of individual trees. We have suggested a weighting scheme where the weights still sum up to one but are no longer equal and may even be negative. Our scheme borrows certain ideas from financial portfolio selection. In particular, we impose an upper bound on the  $L_1$  norm of the vector of weights (that is, on the sum of the absolute values of the weights), which “hedges” the user against weights that are unduly large in absolute value. As a consequence, we call the proposal the *hedged random forest*. It should be noted that our scheme for weighting individual forecasts is of high-level nature and can be applied to any generic forest-combination problem, of which the random forest is essentially a special case.

An extensive empirical analysis has demonstrated that the hedged random forest not only outperforms the standard random forest in terms of forecasting accuracy but also three previous proposals from the machine learning literature for weighted random forests (all of which only employ non-negative weights of the individual trees).

There are (at least) three fruitful avenues for future research. First, study the benefits of the hedged random forest relative to the standard random forest when the latter is not used for forecasting but for other purposes such as the estimation of treatment and causal

effects; for example, see Athey and Imbens (2016) and Wager and Athey (2018). Second, study the possibility of hedging the generalized random forest, which can be used to fit any quantity of interest identified as the solution to a set of local moment equations; see Athey et al. (2019). Third, study the applicability of the hedged random forest for the task of classification, which is related to the task of regression but comes with additional difficulties due to the discontinuous 0-1 loss.



# References

- Athey, S. and Imbens, G. W. (2016). Recursive partitioning for heterogeneous causal effects. *Proceedings of the National Academy of Sciences*, 113(27):7353–7360.
- Athey, S. and Imbens, G. W. (2019). Machine learning methods that economists should know about. *Annual Review of Economics*, 11(1):685–725.
- Athey, S., Tibshirani, J., and Wager, S. (2019). Generalized random forests. *Annals of Statistics*, 47(2):1148–1178.
- Beck, E. and Wolf, M. (2025). Forecasting inflation with the hedged random forest. Working paper, University of Zurich. Available at SSRN: <https://ssrn.com/abstract=5162541>.
- Belloni, A., Chen, D. L., Chernozhukov, V., and Hansen, C. B. (2012). Sparse models and methods for optimal instruments with an application to eminent domain. *Econometrica*, 80(6):2369–2429. Arxiv, 2010.
- Belloni, A., Chernozhukov, V., and Hansen, C. (2014). Inference on treatment effects after selection amongst high-dimensional controls. *Review of Economic Studies*, 81(2):608–650.
- Bergmeir, C. and Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213.
- Bergmeir, C., Hyndman, R. J., and Koo, B. (2018). A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70–83.
- Bodnar, T., Okhrin, O., and Parolya, N. (2019). Optimal shrinkage estimator for high-dimensional mean vector. *Journal of Multivariate Analysis*, 170:63–79.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45:5–32.
- Chan, F. and Mátyás, L. (2022). *Econometrics With Machine Learning*. Springer, Chan (Switzerland).
- Chen, X., Yu, D., and Zhang, X. (2024). Optimal weighted random forests. *Journal of Machine Learning Research*, 25(320):1–81.
- Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., and Robins, J. (2018). Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68.

- DeMiguel, V., Garlappi, L., Nogales, F. J., and Uppal, R. (2009). A generalized approach to portfolio optimization: Improving performance by constraining portfolio norms. *Management Science*, 55(5):798–812.
- Ding, Y. and Zheng, X. (2025). Sub-Gaussian estimation of the scatter matrix in ultra-high dimensional elliptical factor models with  $2 + \epsilon$ th moment. Research Paper 2025-200, HKUST Business School. Available at <https://ssrn.com/abstract=5143723>.
- Dixon, M. F., Halperin, I., and Bilokon, P. (2020). *Machine Learning in Finance*, volume 1170. Springer International Publishing, New York.
- Efron, B. and Hastie, T. (2022). *Computer Age Statistical Inference*. Cambridge University Press, Cambridge.
- Elliott, G. and Timmermann, A. (2016). *Economic Forecasting*. Princeton University Press, Princeton.
- Engle, R. F., Ledoit, O., and Wolf, M. (2019). Large dynamic covariance matrices. *Journal of Business & Economic Statistics*, 37(2):363–375.
- Gamerman, A. and Vovk, V. (2007). Hedging predictions in machine learning. *Computer Journal*, 50(2):151–163.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Hansen, B. E. (2016). The risk of James-Stein and Lasso shrinkage. *Econometric Reviews*, 35(8-10):1456–1470.
- Hansen, C. and Kozbur, D. (2014). Instrumental variables estimation with many weak instruments using regularized JIVE. *Journal of Econometrics*, 182(2):290–308.
- Hansen, C., Kozbur, D., and Misra, S. (2023). Targeted undersmoothing: Sensitivity analysis for sparse estimators. *The Review of Economics and Statistics*, 105(1):101–112.
- Hansen, P. R. and Timmermann, A. (2015). Equivalence between out-of-sample forecast comparisons and Wald statistics. *Econometrica*, 83(6):2485–2505.
- Hastie, T., Tibshirani, R., and Freedman, J. (2017). *The Elements of Statistical Learning*. Springer, New York, second edition.
- Hayashi, F. (2000). *Econometrics*. Princeton University Press, Princeton, New Jersey.

- Jagannathan, R. and Ma, T. (2003). Risk reduction in large portfolios: Why imposing the wrong constraints helps. *Journal of Finance*, 54(4):1651–1684.
- Januschowski, T., Wang, Y., Torkkola, K., Erkkilä, T., Hasson, H., and Gasthaus, J. (2022). Forecasting with trees. *International Journal of Forecasting*, 38(4):1473–1481.
- Jobson, J. D. and Korkie, B. M. (1980). Estimation for Markowitz efficient portfolios. *Journal of the American Statistical Association*, 75:544–554.
- Kelly, B. and Xiu, D. (2023). Financial machine learning. *Foundations and Trends® in Finance*, 13(3–4):205–363.
- Ledoit, O. and Wolf, M. (2021). Shrinkage estimation of large covariance matrices: Keep it simple, statistician? *Journal of Multivariate Analysis*, 186:104796.
- Ledoit, O. and Wolf, M. (2022a). The power of (non-)linear shrinking: A review and guide to covariance matrix estimation. *Journal of Financial Econometrics*, 20(1):187–218.
- Ledoit, O. and Wolf, M. (2022b). Quadratic shrinkage for large covariance matrices. *Bernoulli*, 28(3):1519–1547.
- Liaw, A. and Wiener, M. (2002). Classification and regression by randomforest. *R News*, 2(3):18–22.
- López de Prado, M. M. (2020). *Machine Learning for Asset Managers*. Cambridge University Press, Cambridge.
- Michaud, R. (1989). The Markowitz optimization enigma: Is optimized optimal? *Financial Analysts Journal*, 45:31–42.
- Mullainathan, S. and Spiess, J. (2017). Machine learning: An applied econometric approach. *Journal of Economic Perspectives*, 31(2):87–106.
- Pham, H. and Olafsson, S. (2020). On Cesàro averages for weighted trees in the random forest. *Journal of Classification*, 37(1):223–236.
- Qiu, Y., Xie, T., Yu, J., and Zhang, X. (2020). Mallows-type averaging machine learning techniques. Working paper, Singapore Management University.
- Sancetta, A. (2008). Sample covariance shrinkage for high dimensional dependent data. *Journal of Multivariate Analysis*, 99(5):949–967.

- Timmermann, A. (2006). Forecast combinations. In Elliott, G., Granger, C. W. J., and Timmermann, A., editors, *Handbook of Economic Forecasting*, pages 135–196. North Holland, Amsterdam.
- Varian, H. R. (2014). Big data: New tricks for econometrics. *Journal of Economic Perspectives*, 28(2):3–28.
- Wager, S. and Athey, S. (2018). Estimation and inference of heterogeneous treatment effects using random forests. *Journal of the American Statistical Association*, 113(523):1228–1242.
- Wang, X., Hyndman, R. J., Li, F., and Kang, Y. (2022). Forecast combinations: An over 50-year review. *International Journal of Forecasting*.
- Winham, S. J., Freimuth, R. R., and Biernacka, J. M. (2013). A weighted random forests approach to improve predictive performance. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, 6(6):496–505.
- Wright, M. N. and Ziegler, A. (2017). ranger: A fast implementation of random forests for high dimensional data in C++ and R. *Journal of Statistical Software*, 77(1):1–17.

## A Robustness Checks

The goal of this appendix is to provide robustness checks regarding the nature of the estimator  $\hat{\Sigma}$  and the choice of the gross-exposure constraint  $\kappa$ .

As an alternative estimator  $\hat{\Sigma}$  to nonlinear shrinkage, we will consider the canonical choice, the sample covariance matrix. In terms of  $\kappa$ , we will consider the choices  $\kappa \in \{1, 1.5, 2, 2.5, \infty\}$ , the canonical choice in the forecast-combination literature being  $\kappa = 1$ .

Figure 4 displays the results for all choices of  $\kappa$  when  $\hat{\Sigma}$  is given by nonlinear shrinkage, whereas Figure 5 displays analogous results when nonlinear shrinkage is replaced by the sample covariance matrix.

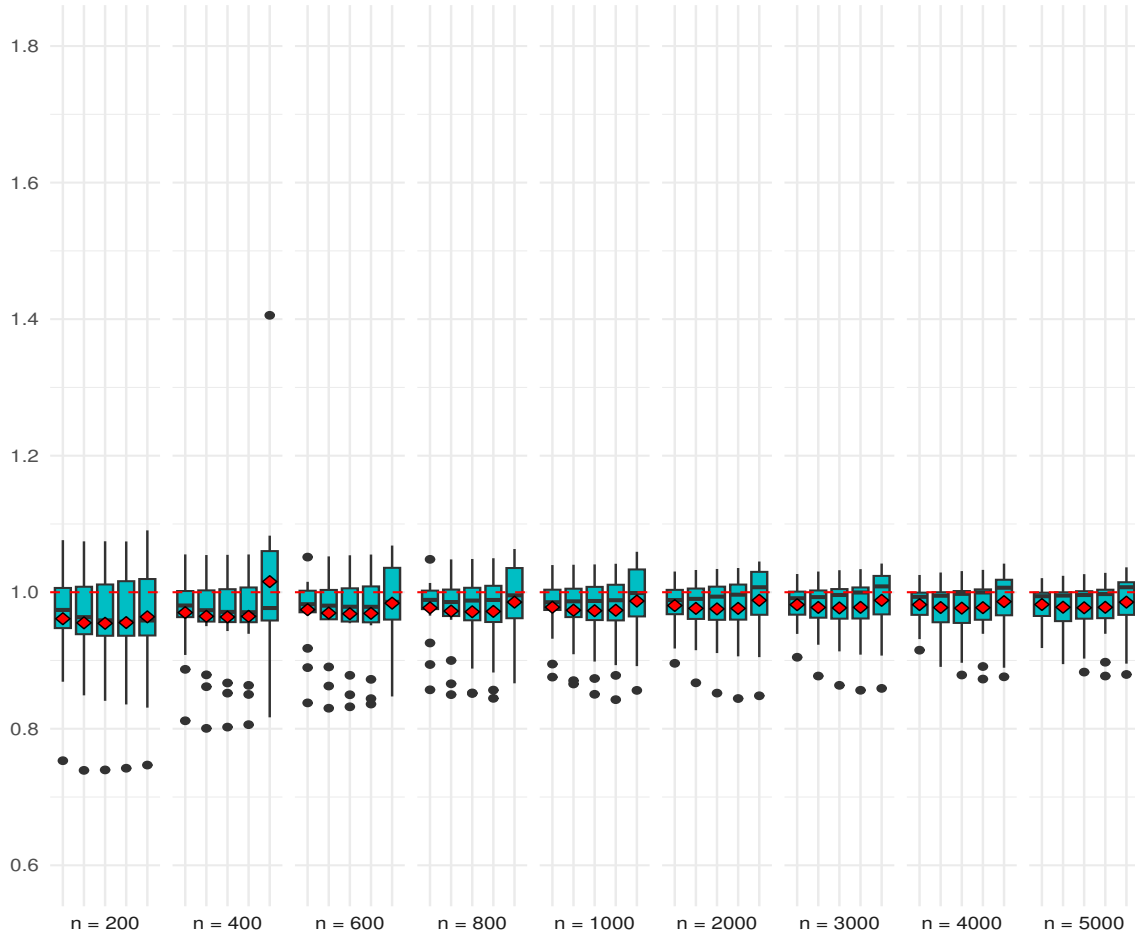


Figure 4: Boxplots of RMSE ratios (4.1). For each training-set size  $n$ , there are five boxplots corresponding to  $\kappa \in \{1, 1.5, 2, 2.5, \infty\}$  where each boxplot is based on the 17 ratios corresponding to the datasets listed in Table 1. The estimator  $\hat{\Sigma}$  is given by nonlinear shrinkage.

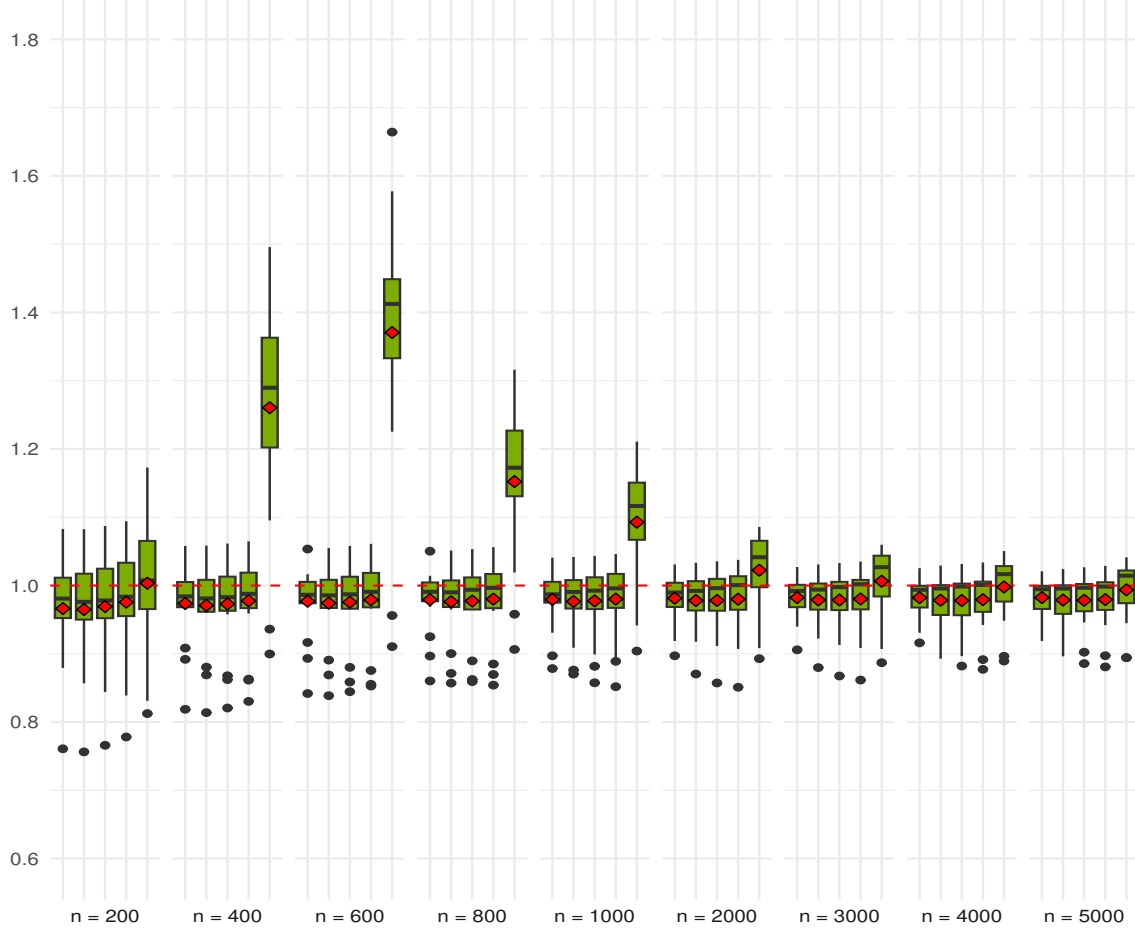


Figure 5: Boxplots of RMSE ratios (4.1). For each training-set size  $n$ , there are five boxplots corresponding to  $\kappa \in \{1, 1.5, 2, 2.5, \infty\}$  where each boxplot is based on the 17 ratios corresponding to the datasets listed in Table 1. The estimator  $\hat{\Sigma}$  is given by the sample covariance matrix.

When  $\hat{\Sigma}$  is given by nonlinear shrinkage, the choice  $\kappa \in \{1.5, 2, 2.5\}$  matters very little, and all three choices perform better than the canonical choice  $\kappa = 1$ . Not imposing a gross-exposure constraint (that is, the choice  $\kappa = \infty$ ) performs worst overall, although it still outperforms RF, on balance.

When  $\hat{\Sigma}$  is given by the sample covariance matrix instead, the choice of  $\kappa$  matters more. In particular, not imposing a gross-exposure constraint (that is, the choice  $\kappa = \infty$ ) can lead to grave underperformance of the HRF when the training-set size,  $n$ , is close to the number of trees,  $p$ . Overall, the choice  $\kappa = 1$  seems to perform best.

Finally, we look at the combined benefit of our version of HRF compared to the *status quo* in the forecast-combination literature: This means that (i) we use nonlinear shrinkage for  $\hat{\Sigma}$  instead of the sample covariance matrix and (ii) we use  $\kappa = 2$  instead of  $\kappa = 1$ . Calling the two versions HRFour and HRFcan (for “our” and “canonical”), we can construct as an

analog to (4.1) the ratio

$$\text{RMSE}_{\text{HRFour}/\text{HRFcan}} := \frac{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{HRFour},b}}}{\sqrt{\frac{1}{B} \sum_{b=1}^B \text{MSE}_{\text{HRFcan},b}}} . \quad (\text{A.1})$$

As an analog to Figure 1 we then obtain Figure 6,

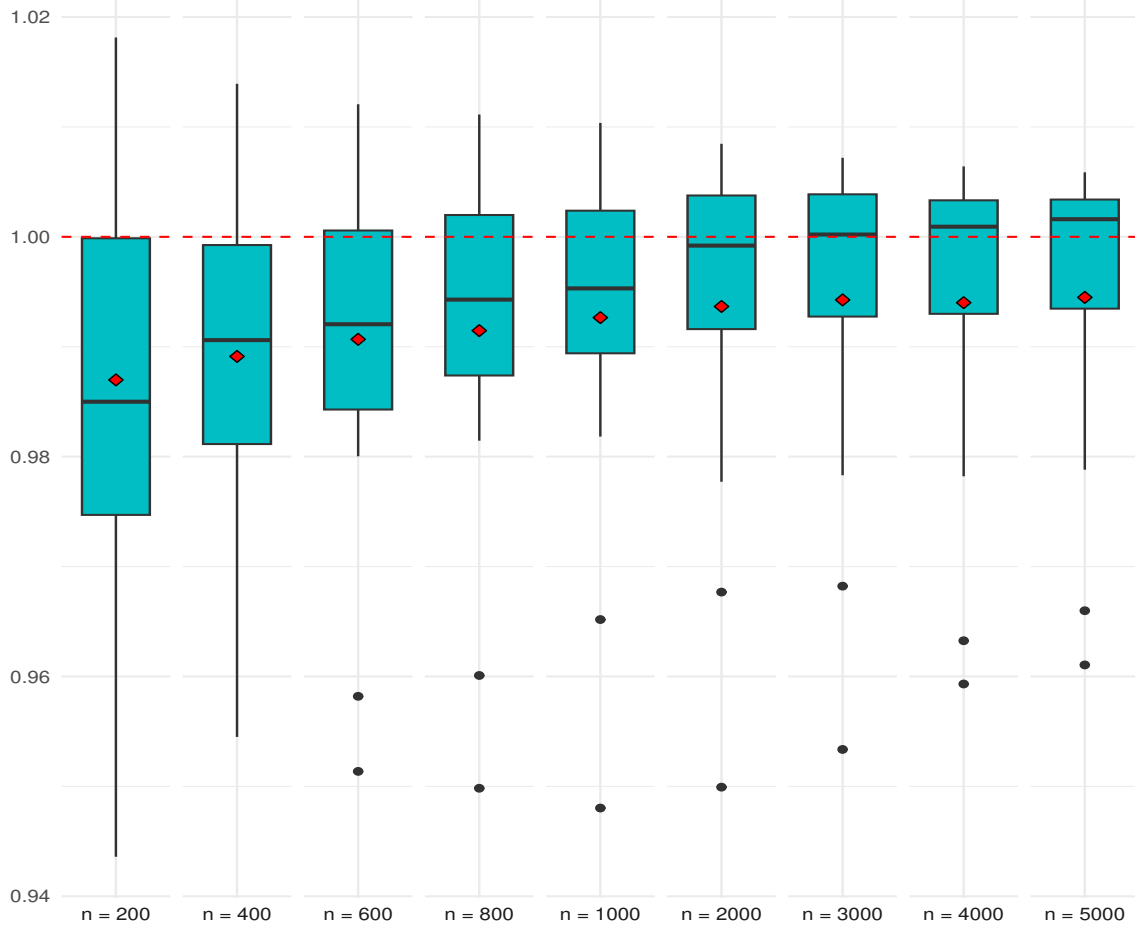


Figure 6: Boxplots of RMSE ratios (A.1). For each training-set size  $n$ , the boxplot is based on the 17 ratios corresponding to the datasets listed in Table 1.

One can see that in terms of the median RMSE ratio, our version outperforms the canonical version for  $n \leq 2000$ , after which the competition becomes a tie, whereas in terms of the mean RMSE ratio, our version outperforms the canonical version for all  $n$ .

## B Detailed Results in Table Format

This appendix presents the RMSE results for all datasets, training-set sizes, and forecasting methods discussed in Sections 4.2 and 4.4.1, including the Ridge, 2-Step Ridge, and HRF-MinVar benchmarks of Section 4.4.1.

| Name                    | RF          | HRF         | WRF         | CRF           | Ridge       | 2-Step Ridge | HRF-MinVar   |
|-------------------------|-------------|-------------|-------------|---------------|-------------|--------------|--------------|
| Ailerons                | 0.23        | 0.22        | 0.23        | 0.22          | 0.23        | 0.24         | <b>0.22</b>  |
| Bike_Sharing_Demand     | 126.06      | 125.55      | 125.41      | <b>123.91</b> | 135.05      | 143.11       | 125.51       |
| brazilian_houses        | 0.17        | 0.14        | 0.16        | 0.16          | <b>0.14</b> | 0.15         | 0.14         |
| cpu_act                 | 4.21        | 3.65        | 3.75        | 3.66          | 4.06        | 5.06         | <b>3.65</b>  |
| diamonds                | <b>0.26</b> | 0.28        | 0.26        | 0.26          | 0.31        | 0.36         | 0.28         |
| elevators               | 4.78        | 4.72        | 4.77        | 4.75          | 4.90        | 5.33         | <b>4.72</b>  |
| house_16H               | <b>0.70</b> | 0.72        | 0.70        | 0.70          | 0.74        | 0.78         | 0.72         |
| house_sales             | 0.26        | 0.25        | 0.26        | 0.26          | 0.27        | 0.29         | <b>0.25</b>  |
| houses                  | 0.37        | 0.36        | 0.37        | 0.36          | 0.37        | 0.39         | <b>0.36</b>  |
| isolet                  | 5.68        | <b>5.32</b> | 5.64        | 5.50          | 5.47        | 6.04         | 5.32         |
| medical_charges         | 0.16        | <b>0.12</b> | 0.13        | 0.13          | 0.13        | 0.15         | 0.12         |
| MiamiHousing2016        | 0.28        | 0.26        | 0.27        | 0.27          | 0.27        | 0.29         | <b>0.26</b>  |
| nyc-taxi-green-dec-2016 | 0.50        | 0.52        | 0.50        | <b>0.50</b>   | 0.56        | 0.61         | 0.52         |
| pol                     | 16.60       | 13.99       | 15.97       | 15.20         | 14.17       | 16.48        | <b>13.98</b> |
| sulfur                  | 0.04        | <b>0.03</b> | 0.04        | 0.03          | 0.04        | 0.04         | 0.03         |
| superconduct            | 17.37       | 17.57       | 17.34       | <b>17.31</b>  | 18.18       | 20.36        | 17.55        |
| wine_quality            | 0.74        | 0.77        | <b>0.74</b> | 0.75          | 0.81        | 0.87         | 0.77         |

Table 4: Results for all data sets with  $n = 200$ . Values for the Ailerons and elevators data sets are scaled by a factor of 1,000 for better readability.

| Name                    | RF          | HRF         | WRF         | CRF           | Ridge  | 2-Step Ridge | HRF-MinVar   |
|-------------------------|-------------|-------------|-------------|---------------|--------|--------------|--------------|
| Ailerons                | 0.21        | 0.20        | 0.21        | 0.21          | 0.28   | 0.26         | <b>0.20</b>  |
| Bike_Sharing_Demand     | 119.70      | 118.88      | 119.19      | <b>117.58</b> | 169.82 | 152.08       | 118.89       |
| brazilian_houses        | 0.15        | 0.13        | 0.14        | 0.14          | 0.15   | 0.15         | <b>0.13</b>  |
| cpu_act                 | 3.17        | 3.07        | 3.09        | 3.07          | 4.67   | 4.64         | <b>3.07</b>  |
| diamonds                | <b>0.26</b> | 0.27        | 0.26        | 0.26          | 0.40   | 0.38         | 0.27         |
| elevators               | 4.42        | 4.30        | 4.41        | 4.38          | 5.66   | 5.56         | <b>4.30</b>  |
| house_16H               | <b>0.68</b> | 0.71        | 0.68        | 0.69          | 0.85   | 0.87         | 0.71         |
| house_sales             | 0.24        | 0.24        | 0.24        | 0.24          | 0.32   | 0.31         | <b>0.24</b>  |
| houses                  | 0.34        | 0.33        | 0.34        | 0.33          | 0.46   | 0.41         | <b>0.33</b>  |
| isolet                  | 4.83        | <b>4.57</b> | 4.79        | 4.68          | 5.80   | 5.77         | 4.57         |
| medical_charges         | 0.13        | <b>0.11</b> | 0.11        | 0.12          | 0.16   | 0.16         | 0.11         |
| MiamiHousing2016        | 0.24        | 0.23        | 0.24        | 0.24          | 0.31   | 0.29         | <b>0.23</b>  |
| nyc-taxi-green-dec-2016 | 0.49        | 0.51        | 0.49        | <b>0.49</b>   | 0.69   | 0.67         | 0.51         |
| pol                     | 13.20       | 11.32       | 12.24       | 12.42         | 13.35  | 14.03        | <b>11.31</b> |
| sulfur                  | 0.03        | 0.03        | 0.03        | <b>0.03</b>   | 0.04   | 0.04         | 0.03         |
| superconduct            | 15.66       | 15.70       | 15.63       | <b>15.59</b>  | 17.97  | 24.22        | 15.70        |
| wine_quality            | 0.72        | 0.74        | <b>0.72</b> | 0.72          | 1.02   | 0.97         | 0.74         |

Table 5: Results for all data sets with  $n = 400$ . Values for the Ailerons and elevators data sets are scaled by a factor of 1,000 for better readability.



| Name                    | RF          | HRF         | WRF         | CRF           | Ridge  | 2-Step Ridge | HRF-MinVar   |
|-------------------------|-------------|-------------|-------------|---------------|--------|--------------|--------------|
| Ailerons                | 0.20        | 0.20        | 0.20        | 0.20          | 0.26   | 0.30         | <b>0.20</b>  |
| Bike_Sharing_Demand     | 116.30      | 115.64      | 115.88      | <b>114.33</b> | 180.73 | 177.29       | 115.61       |
| brazilian_houses        | 0.14        | 0.12        | 0.14        | 0.14          | 0.14   | 0.18         | <b>0.12</b>  |
| cpu_act                 | 2.96        | 2.92        | 2.93        | 2.92          | 4.33   | 5.30         | <b>2.92</b>  |
| diamonds                | <b>0.26</b> | 0.26        | 0.26        | 0.26          | 0.37   | 0.50         | 0.26         |
| elevators               | 4.24        | <b>4.08</b> | 4.23        | 4.19          | 5.33   | 6.51         | 4.08         |
| house_16H               | <b>0.66</b> | 0.70        | 0.66        | 0.67          | 0.83   | 0.99         | 0.70         |
| house_sales             | 0.23        | 0.23        | 0.23        | <b>0.23</b>   | 0.30   | 0.35         | 0.23         |
| houses                  | 0.33        | 0.31        | 0.33        | 0.32          | 0.42   | 0.47         | <b>0.31</b>  |
| isolet                  | 4.42        | 4.24        | 4.39        | 4.31          | 5.36   | 6.28         | <b>4.24</b>  |
| medical_charges         | 0.12        | <b>0.10</b> | 0.11        | 0.11          | 0.15   | 0.18         | 0.10         |
| MiamiHousing2016        | 0.23        | 0.22        | 0.23        | 0.23          | 0.30   | 0.33         | <b>0.22</b>  |
| nyc-taxi-green-dec-2016 | 0.49        | 0.50        | 0.49        | <b>0.48</b>   | 0.67   | 0.80         | 0.50         |
| pol                     | 11.81       | 10.07       | 10.91       | 11.16         | 14.59  | 14.34        | <b>10.07</b> |
| sulfur                  | 0.03        | 0.03        | 0.03        | <b>0.03</b>   | 0.04   | 0.05         | 0.03         |
| superconduct            | 14.79       | 14.85       | 14.77       | <b>14.73</b>  | 17.27  | 35.31        | 14.84        |
| wine_quality            | 0.71        | 0.72        | <b>0.71</b> | 0.71          | 1.03   | 1.15         | 0.72         |

Table 6: Results for all data sets with  $n = 600$ . Values for the Ailerons and elevators data sets are scaled by a factor of 1,000 for better readability.

| Name                    | RF          | HRF         | WRF         | CRF           | Ridge  | 2-Step Ridge | HRF-MinVar  |
|-------------------------|-------------|-------------|-------------|---------------|--------|--------------|-------------|
| Ailerons                | 0.20        | 0.19        | 0.19        | 0.19          | 0.20   | 0.41         | <b>0.19</b> |
| Bike_Sharing_Demand     | 114.41      | 113.88      | 114.06      | <b>112.62</b> | 134.02 | 243.17       | 113.84      |
| brazilian_houses        | 0.13        | 0.12        | 0.13        | 0.13          | 0.12   | 0.21         | <b>0.12</b> |
| cpu_act                 | 2.88        | 2.85        | 2.86        | 2.86          | 3.07   | 7.20         | <b>2.85</b> |
| diamonds                | <b>0.25</b> | 0.26        | 0.25        | 0.25          | 0.26   | 0.65         | 0.26        |
| elevators               | 4.11        | <b>3.95</b> | 4.11        | 4.06          | 4.12   | 9.03         | 3.95        |
| house_16H               | <b>0.65</b> | 0.68        | 0.65        | 0.66          | 0.69   | 1.33         | 0.68        |
| house_sales             | 0.23        | 0.22        | 0.23        | <b>0.22</b>   | 0.23   | 0.49         | 0.22        |
| houses                  | 0.32        | 0.30        | 0.32        | 0.31          | 0.31   | 0.64         | <b>0.30</b> |
| isolet                  | 4.17        | <b>4.03</b> | 4.15        | 4.08          | 4.18   | 8.16         | 4.03        |
| medical_charges         | 0.12        | <b>0.10</b> | 0.10        | 0.11          | 0.11   | 0.25         | 0.10        |
| MiamiHousing2016        | 0.22        | 0.21        | 0.22        | 0.22          | 0.23   | 0.43         | <b>0.21</b> |
| nyc-taxi-green-dec-2016 | 0.48        | 0.49        | 0.48        | <b>0.48</b>   | 0.52   | 1.10         | 0.49        |
| pol                     | 10.97       | <b>9.37</b> | 10.23       | 10.40         | 10.60  | 17.69        | 9.37        |
| sulfur                  | 0.03        | 0.03        | 0.03        | <b>0.03</b>   | 0.03   | 0.06         | 0.03        |
| superconduct            | 14.25       | 14.29       | 14.23       | <b>14.19</b>  | 15.30  | 36.84        | 14.29       |
| wine_quality            | 0.70        | 0.71        | <b>0.70</b> | 0.70          | 0.74   | 1.55         | 0.71        |

Table 7: Results for all data sets with  $n = 800$ . Values for the Ailerons and elevators data sets are scaled by a factor of 1,000 for better readability.

| Name                    | RF          | HRF         | WRF         | CRF           | Ridge       | 2-Step Ridge | HRF-MinVar  |
|-------------------------|-------------|-------------|-------------|---------------|-------------|--------------|-------------|
| Ailerons                | 0.19        | <b>0.19</b> | 0.19        | 0.19          | 0.19        | 3.26         | 0.19        |
| Bike_Sharing_Demand     | 112.91      | 112.60      | 112.60      | <b>111.26</b> | 123.07      | 1605.67      | 112.55      |
| brazilian_houses        | 0.13        | 0.11        | 0.12        | 0.12          | <b>0.11</b> | 1.04         | 0.11        |
| cpu_act                 | 2.82        | <b>2.80</b> | 2.81        | 2.80          | 2.88        | 34.08        | 2.80        |
| diamonds                | <b>0.25</b> | 0.26        | 0.25        | 0.25          | 0.26        | 3.41         | 0.26        |
| elevators               | 4.00        | <b>3.82</b> | 4.00        | 3.95          | 3.87        | 73.08        | 3.83        |
| house_16H               | <b>0.65</b> | 0.67        | 0.65        | 0.65          | 0.68        | 11.62        | 0.67        |
| house_sales             | 0.22        | 0.22        | 0.22        | <b>0.22</b>   | 0.22        | 4.00         | 0.22        |
| houses                  | 0.31        | 0.30        | 0.31        | 0.30          | 0.30        | 6.16         | <b>0.30</b> |
| isolet                  | 4.01        | <b>3.88</b> | 3.99        | 3.92          | 3.95        | 72.84        | 3.88        |
| medical_charges         | 0.11        | <b>0.10</b> | 0.10        | 0.10          | 0.10        | 1.44         | 0.10        |
| MiamiHousing2016        | 0.21        | 0.20        | 0.21        | 0.21          | 0.21        | 3.15         | <b>0.20</b> |
| nyc-taxi-green-dec-2016 | 0.48        | 0.49        | 0.48        | <b>0.48</b>   | 0.50        | 11.02        | 0.49        |
| pol                     | 10.21       | 8.76        | 9.56        | 9.72          | 9.43        | 69.64        | <b>8.76</b> |
| sulfur                  | 0.03        | 0.03        | 0.03        | <b>0.03</b>   | 0.03        | 0.35         | 0.03        |
| superconduct            | 13.81       | 13.92       | 13.80       | <b>13.77</b>  | 14.47       | 106.94       | 13.92       |
| wine_quality            | 0.69        | 0.70        | <b>0.69</b> | 0.69          | 0.71        | 6.61         | 0.70        |

Table 8: Results for all data sets with  $n = 1000$ . Values for the Ailerons and elevators data sets are scaled by a factor of 1,000 for better readability.

| Name                    | RF          | HRF         | WRF         | CRF           | Ridge       | 2-Step Ridge | HRF-MinVar  |
|-------------------------|-------------|-------------|-------------|---------------|-------------|--------------|-------------|
| Ailerons                | 0.18        | 0.18        | 0.18        | <b>0.18</b>   | 0.18        | 0.20         | 0.18        |
| Bike_Sharing_Demand     | 109.05      | 109.00      | 108.86      | <b>107.82</b> | 112.11      | 131.93       | 108.96      |
| brazilian_houses        | 0.12        | 0.11        | 0.11        | 0.12          | <b>0.11</b> | 0.12         | 0.11        |
| cpu_act                 | 2.66        | 2.66        | 2.65        | <b>2.65</b>   | 2.67        | 2.97         | 2.66        |
| diamonds                | <b>0.25</b> | 0.25        | 0.25        | 0.25          | 0.25        | 0.25         | 0.25        |
| elevators               | 3.70        | 3.49        | 3.69        | 3.63          | <b>3.48</b> | 3.84         | 3.50        |
| house_16H               | <b>0.63</b> | 0.65        | 0.63        | 0.63          | 0.65        | 0.73         | 0.65        |
| house_sales             | 0.21        | 0.21        | 0.21        | <b>0.21</b>   | 0.21        | 0.23         | 0.21        |
| houses                  | 0.29        | 0.28        | 0.29        | 0.28          | 0.28        | 0.30         | <b>0.28</b> |
| isolet                  | 3.59        | <b>3.50</b> | 3.57        | 3.52          | 3.52        | 3.99         | 3.50        |
| medical_charges         | 0.10        | <b>0.10</b> | 0.10        | 0.10          | 0.10        | 0.10         | 0.10        |
| MiamiHousing2016        | 0.19        | 0.19        | 0.19        | 0.19          | 0.19        | 0.22         | <b>0.19</b> |
| nyc-taxi-green-dec-2016 | 0.47        | 0.47        | 0.47        | <b>0.47</b>   | 0.48        | 0.49         | 0.47        |
| pol                     | 8.39        | 7.17        | 7.85        | 7.98          | 7.24        | 9.69         | <b>7.17</b> |
| sulfur                  | 0.03        | 0.03        | 0.03        | 0.03          | 0.03        | 0.03         | <b>0.03</b> |
| superconduct            | 12.60       | 12.69       | 12.59       | <b>12.57</b>  | 12.84       | 17.56        | 12.69       |
| wine_quality            | 0.66        | 0.67        | <b>0.66</b> | 0.66          | 0.67        | 0.70         | 0.67        |

Table 9: Results for all data sets with  $n = 2000$ . Values for the Ailerons and elevators data sets are scaled by a factor of 1,000 for better readability.

| Name                    | RF          | HRF    | WRF         | CRF           | Ridge       | 2-Step Ridge | HRF-MinVar  |
|-------------------------|-------------|--------|-------------|---------------|-------------|--------------|-------------|
| Ailerons                | 0.18        | 0.18   | 0.18        | <b>0.17</b>   | 0.18        | 0.18         | 0.18        |
| Bike_Sharing_Demand     | 106.88      | 106.94 | 106.74      | <b>105.89</b> | 108.77      | 117.15       | 106.90      |
| brazilian_houses        | 0.11        | 0.10   | 0.11        | 0.11          | <b>0.10</b> | 0.11         | 0.10        |
| cpu_act                 | 2.57        | 2.57   | 2.57        | <b>2.56</b>   | 2.57        | 2.74         | 2.57        |
| diamonds                | <b>0.25</b> | 0.25   | 0.25        | 0.25          | 0.25        | 0.25         | 0.25        |
| elevators               | 3.52        | 3.31   | 3.51        | 3.45          | <b>3.29</b> | 3.52         | 3.32        |
| house_16H               | <b>0.62</b> | 0.63   | 0.62        | 0.62          | 0.63        | 0.69         | 0.63        |
| house_sales             | 0.21        | 0.21   | 0.21        | <b>0.20</b>   | 0.21        | 0.21         | 0.21        |
| houses                  | 0.28        | 0.27   | 0.28        | 0.27          | 0.27        | 0.28         | <b>0.27</b> |
| isolet                  | 3.38        | 3.30   | 3.36        | 3.32          | 3.32        | 3.65         | <b>3.30</b> |
| medical_charges         | 0.10        | 0.09   | 0.10        | 0.10          | 0.10        | 0.10         | <b>0.09</b> |
| MiamiHousing2016        | 0.18        | 0.18   | 0.18        | 0.18          | 0.18        | 0.20         | <b>0.18</b> |
| nyc-taxi-green-dec-2016 | 0.46        | 0.47   | 0.46        | <b>0.46</b>   | 0.47        | 0.48         | 0.47        |
| pol                     | 7.41        | 6.43   | 6.97        | 7.06          | 6.44        | 7.94         | <b>6.43</b> |
| sulfur                  | 0.02        | 0.02   | 0.02        | 0.02          | 0.02        | 0.03         | <b>0.02</b> |
| superconduct            | 11.92       | 12.01  | 11.91       | <b>11.89</b>  | 12.07       | 14.62        | 12.01       |
| wine_quality            | 0.64        | 0.64   | <b>0.64</b> | 0.64          | 0.64        | 0.68         | 0.64        |

Table 10: Results for all data sets with  $n = 3000$ . Values for the Ailerons and elevators data sets are scaled by a factor of 1,000 for better readability.

| Name                    | RF          | HRF    | WRF         | CRF           | Ridge       | 2-Step Ridge | HRF-MinVar  |
|-------------------------|-------------|--------|-------------|---------------|-------------|--------------|-------------|
| Ailerons                | 0.17        | 0.17   | 0.17        | <b>0.17</b>   | 0.17        | 0.18         | 0.17        |
| Bike_Sharing_Demand     | 105.63      | 105.82 | 105.52      | <b>104.78</b> | 107.03      | 112.17       | 105.79      |
| brazilian_houses        | 0.11        | 0.10   | 0.10        | 0.10          | <b>0.10</b> | 0.11         | 0.10        |
| cpu_act                 | 2.51        | 2.50   | 2.51        | <b>2.50</b>   | 2.50        | 2.65         | 2.50        |
| diamonds                | <b>0.25</b> | 0.25   | 0.25        | 0.25          | 0.25        | 0.25         | 0.25        |
| elevators               | 3.40        | 3.19   | 3.39        | 3.33          | <b>3.18</b> | 3.35         | 3.20        |
| house_16H               | <b>0.60</b> | 0.62   | 0.60        | 0.61          | 0.62        | 0.67         | 0.62        |
| house_sales             | 0.20        | 0.20   | 0.20        | <b>0.20</b>   | 0.20        | 0.21         | 0.20        |
| houses                  | 0.27        | 0.26   | 0.27        | 0.27          | 0.26        | 0.27         | <b>0.26</b> |
| isolet                  | 3.26        | 3.18   | 3.24        | 3.20          | 3.19        | 3.49         | <b>3.18</b> |
| medical_charges         | 0.10        | 0.09   | 0.09        | <b>0.09</b>   | 0.09        | 0.09         | 0.09        |
| MiamiHousing2016        | 0.17        | 0.17   | 0.17        | 0.17          | 0.17        | 0.19         | <b>0.17</b> |
| nyc-taxi-green-dec-2016 | 0.46        | 0.46   | 0.46        | <b>0.46</b>   | 0.46        | 0.47         | 0.46        |
| pol                     | 6.71        | 5.89   | 6.33        | 6.41          | 5.89        | 7.09         | <b>5.89</b> |
| sulfur                  | 0.02        | 0.02   | 0.02        | 0.02          | 0.02        | 0.03         | <b>0.02</b> |
| superconduct            | 11.49       | 11.58  | 11.49       | <b>11.47</b>  | 11.61       | 13.53        | 11.58       |
| wine_quality            | 0.62        | 0.63   | <b>0.62</b> | 0.62          | 0.62        | 0.66         | 0.62        |

Table 11: Results for all data sets with  $n = 4000$ . Values for the Ailerons and elevators data sets are scaled by a factor of 1,000 for better readability.

| Name                    | RF          | HRF    | WRF         | CRF           | Ridge       | 2-Step Ridge | HRF-MinVar  |
|-------------------------|-------------|--------|-------------|---------------|-------------|--------------|-------------|
| Ailerons                | 0.17        | 0.17   | 0.17        | <b>0.17</b>   | 0.17        | 0.18         | 0.17        |
| Bike_Sharing_Demand     | 104.55      | 104.76 | 104.45      | <b>103.81</b> | 105.74      | 110.13       | 104.73      |
| brazilian_houses        | 0.10        | 0.09   | 0.10        | 0.10          | <b>0.09</b> | 0.10         | 0.09        |
| cpu_act                 | 2.47        | 2.46   | 2.46        | <b>2.46</b>   | 2.46        | 2.59         | 2.46        |
| diamonds                | <b>0.24</b> | 0.25   | 0.24        | 0.25          | 0.25        | 0.25         | 0.25        |
| elevators               | 3.31        | 3.11   | 3.30        | 3.23          | <b>3.09</b> | 3.26         | 3.11        |
| house_16H               | 0.59        | 0.61   | <b>0.59</b> | 0.59          | 0.61        | 0.66         | 0.61        |
| house_sales             | 0.20        | 0.20   | 0.20        | <b>0.20</b>   | 0.20        | 0.21         | 0.20        |
| houses                  | 0.27        | 0.26   | 0.27        | 0.26          | 0.26        | 0.27         | <b>0.26</b> |
| isolet                  | 3.16        | 3.09   | 3.14        | 3.10          | 3.10        | 3.39         | <b>3.09</b> |
| medical_charges         | 0.10        | 0.09   | 0.09        | <b>0.09</b>   | 0.09        | 0.09         | 0.09        |
| MiamiHousing2016        | 0.17        | 0.17   | 0.17        | 0.17          | 0.17        | 0.18         | <b>0.17</b> |
| nyc-taxi-green-dec-2016 | 0.46        | 0.45   | 0.45        | <b>0.45</b>   | 0.45        | 0.46         | 0.45        |
| pol                     | 6.26        | 5.54   | 5.94        | 6.00          | <b>5.53</b> | 6.62         | 5.54        |
| sulfur                  | 0.02        | 0.02   | 0.02        | 0.02          | 0.02        | 0.02         | <b>0.02</b> |
| superconduct            | 11.13       | 11.21  | 11.13       | <b>11.11</b>  | 11.24       | 12.86        | 11.21       |
| wine_quality            | 0.60        | 0.61   | <b>0.60</b> | 0.60          | 0.61        | 0.65         | 0.61        |

Table 12: Results for all data sets with  $n = 5000$ . Values for the Ailerons and elevators data sets are scaled by a factor of 1,000 for better readability.